

第 3 章：How You Run Programs (如何运行程序)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：第一部分 (Getting Started, 入门) 的收官章。上一章讲清了"Python 如何运行" (解释器模型)，本章讲实操——你在哪里、用什么方式把代码喂给解释器。全部围绕一个主题：启动 (launching)。学完本章，你将能从命令行、IDLE、IDE、手机 App 甚至浏览器里运行 Python，并第一次接触模块 (module) 与 import——这是理解 Python 程序架构的钥匙，全书第二部分起将正式开始语言学习。

3.1 开篇引言

英文原文

OK, it's time to start running some code. Now that you have a handle on Python's purpose and execution model, you're finally ready to start some real Python programming. There are multiple ways to tell Python to execute the code you type, and this chapter covers all the major program launching techniques in comm

on use today. Along the way, you'll learn both how to run code interactively, and how to save it in files to be run using a variety of techniques—with command lines, icon clicks, the IDLE GUI, mobile apps, web-based interfaces, module imports, and more. As for the previous chapter, if you have prior programming experience and are anxious to start digging into Python itself, you may want to skim this chapter and move on to Chapter 4. But don't skip this chapter's early coverage of preliminaries and conventions, its overview of debugging techniques, or its first look at module imports—a topic essential to understanding Python's program architecture, which we won't revisit until Part V. It's also worthwhile to browse the sections on IDEs and apps, to sample tools that may be more useful once you start coding larger programs.

中文翻译

好了，是时候开始运行一些代码了。既然你已经掌握了 Python 的用途和执行模型，你终于准备好开始真正的 Python 编程了。有多种方式可以让 Python 执行你输入的代码，本章将覆盖当今实践中所有主要的程序启动技术。在此过程中，你既会学到如何交互式（interactively）运行代码，也会学到如何把代码保存到文件中（files），然后用各种技术来运行它——包括命令行（command lines）、点击图标（icon clicks）、IDLE 图形界面（GUI）、移动应用（mobile apps）、网页接口（web-based interfaces）、模块导入（module imports）等等。和上一章一样，如果你有编

程经验、急着想深入 Python 本身，你可以略读本章直接跳到第 4 章。但请不要跳过本章前面的预备知识和约定（preliminaries and conventions）、调试技术概述，以及第一次对模块导入（imports）的接触——后者对于理解 Python 的程序架构至关重要，而我们要到第五部分才会再次回到它。另外也值得浏览一下关于 IDE 和应用的章节，为将来编写更大程序时可能更有用的工具做个采样。

深度理解

- 核心概念：本章只有一个主题——"启动"（launching）。Python 是解释型语言，同样的代码可以通过 N 种方式"喂"给解释器：交互式输入、命令行文件、点击图标、GUI/IDE、App、浏览器、import。理解这些方式的异同，是你搭建开发环境的第一课。

- 为什么先谈运行再谈语法：这是教学顺序的精心设计——第 1 章讲"为什么学"，第 2 章讲"解释器怎么工作"，第 3 章讲"怎么把代码跑起来"，第 4 章起才正式讲语言本身。就像学开车前先学会"钥匙插哪儿、引擎在哪"。

- 本章对你（初学者）的真正价值：作者三位一体地埋了三个伏笔——①调试技术概述；②import 与模块（Part V 才展开）；③IDE/App 工具全景。作者明确说这些"不能跳过"，因为它们决定了你能不能在后续章节"跟着跑示例"。

- 生活类比：学做饭前，你得先知道自家的炉子怎么点火（煤气灶/电磁炉/微波炉）、锅放哪。本章就是"熟悉自家厨房"的一章——语言本身（菜谱）从第 4 章才正式开讲。

·初学者误区：很多新手跳过运行细节直接写代码，结果连“为什么我的脚本双击没反应”“为什么命令行报'python'不是内部或外部命令”都搞不清。本章正是消灭这些“环境劝退题”的。

3.2 安装 Python (Installing Python)

英文原文

This book will generally assume that you have access to a recent version of Python on your computer, tablet, or phone. Python installation is not required for this book, and isn't necessary in some contexts, but you'll need a Python to work along with examples and do end-of-part exercises, and both are great ways to make concepts more concrete. If you don't have a Python and wish to set one up, see Appendix A now for per-platform install help. In short:- Windows and macOS users fetch and run a self-installing executable file that puts Python on their devices. Simply double-click and say Yes or Next at all prompts.- Linux (including Windows WSL) users may have a usable Python preinstalled on their computers, but can install one if needed or desired from their distribution's repositories.- Android and iOS users install an app that allows them to run Python locally on their phones and tablets.- Unix (and some Linux) users often compile Python from its full source code distribution package. For ex

ample, smartphone users fetch a Python app at an app store, and Windows and macOS PC users get an installer at the downloads page of Python's main website. Python may also be had through other distribution channels, and some Python coding modes covered ahead, such as Jupyter notebooks, have unique install and usage steps. You may not be able to select a Python version in some contexts, but this book uses Python 3.12, so closer to that is better. Before you install, you should generally check to see if Python is already present. Look for it on Windows in the Start menu, and on most platforms by running a Python command line as described both in Appendix A and this chapter's next section. To see how, let's take installs as a given, and move on to learning the many ways to run Python code.

中文翻译

本书通常假设你在电脑、平板或手机上可以访问到较新版本的 Python。本书并不强制要求安装 Python，在某些环境中也非必要——但你要跟着示例动手实践、做每个部分末尾的练习，就确实需要一个 Python；而这两者都是让概念更具体的绝佳方式（great ways to make concepts more concrete）。如果你还没有 Python 并且想装一个，请现在就去查附录 A 获取各平台的安装帮助。简而言之：- Windows 和 macOS 用户下载并运行一个自安装可执行文件，把 Python 装到设备上。只需双击，并在所有提示处选择"Yes"或"Next"。- Linux（包括 Windows WSL）用户电脑

上可能已经预装了可用的 Python；如果需要或想要，也可以从发行版仓库 (repositories) 安装。- Android 和 iOS 用户安装一个可在手机和平板上本地运行 Python 的应用。- Unix (以及部分 Linux) 用户常常从完整的源码发行包编译 Python。例如，智能手机用户在应用商店获取一个 Python 应用，而 Windows 和 macOS 电脑用户则在 Python 官网的下载页面 (downloads page) 获取安装程序。Python 也有其他分发渠道；有些本书后面会讲到的编码模式，如 Jupyter 笔记本，有独特的安装和使用步骤。在某些环境中你可能无法选择 Python 版本，但本书使用的是 Python 3.12，越接近这个版本越好。安装之前，你通常应该先检查 Python 是否已经存在。在 Windows 上可以在开始菜单中查找，在大多数平台上则按附录 A 和本章下一节所述运行一个 Python 命令行来检查。为了看清怎么做，我们先把“已安装”当作既定条件，接着去学习运行 Python 代码的诸多方式。

深度理解

·核心概念：Python 的“安装”在各平台形态完全不同——从 Windows 的图形安装包、Linux 的包管理器、移动端 App，到 Unix 的从源码编译。这是开源解释型语言特有的“分发自由”。

·为什么有这么多形态：因为 Python 本质是源代码，需要运行时（解释器）。它不像手机 App 那样“装一个捆绑包”，而是“装一个解释器 + 一门语言”。不同平台生态决定了不同安装方式。

·关键提示——先查再装：本章反复强调“先看系统里有没

有 Python"。Windows 上 python 命令可能指向 Microsoft Store 的假入口；macOS 自带的可能是旧版 Python 2。在动手下载前，先跑一下 `python --version`。

·版本选择：本书用 3.12。不同 Python 3.x 之间 (3.9/3.11/3.12/3.13) 语法基本兼容，但个别行为有差异。选择 3.10+ 以上版本是当下稳妥的选择。

·生活类比：装 Python 就像搬进一套"自带基本家具但没有厨房电器"的公寓——你要按自家房型（平台）选购电器（不同安装方式），住进来前先确认房东有没有留旧电器（预装 Python）。

·初学者误区：以为装完 Python 就有"图标"可点。其实 Python 是命令行工具，真正干活靠两种界面：交互式命令行 (REPL) 和脚本文件。装完没图形入口是正常的。

3.3 交互式代码 (Interactive Code)

英文原文

This section gets us started with the basics of interactive Python coding. Because it's our first look at running code, we also cover some startup logistics here, such as setting up a working directory, so be sure to read this section first if you're relatively new to programming. This section also explains some conventions used throughout the book, so most readers should probably take at least a quick look here.

中文翻译

本节让我们从交互式 Python 编程的基础开始。因为这是我们第一次接触运行代码，我们也会在这里讲一些启动相关的"后勤"事宜，比如建立工作目录（working directory），所以如果你是编程新手，请务必先读这一节。本节还解释了全书通篇使用的一些约定（conventions），所以大多数读者都值得至少快速浏览一下。

深度理解

- 核心概念：交互模式（interactive mode）= "输入一行、立即执行、立即看结果"。它不保存代码，是试验场而非工作间。
- 为什么要先讲"后勤"：Lutz 用一整节讲工作目录、提示符约定等"小事"。因为这些小事 80% 的新手都会踩坑——尤其是"要不要把 \$ 和 >>> 打进去"这个问题。
- 生活类比：交互模式就像自言自语："2 的 8 次方是多少？"——问完就有答，但说过就忘。它适合验证想法，不适合留存成果。

3.3.1 启动交互式 REPL (Starting an Interactive REPL)

英文原文

By most measures, the simplest way to run Python code is to type it at Python's interactive command line, sometimes called the interactive prompt, and often

more concisely labeled the REPL (which stands for read–eval–print loop). There are a variety of ways to start this command line—in a coding GUI or app, in a web notebook, from a system console, and so on. Assuming Python is installed locally as an executable program on your device, the most platform-neutral way to start an interactive interpreter session is to simply type a Python command at your device's console prompt, without any arguments. Since this is most common, nearly universal, and arguably simplest, let's start here. Despite this scheme's generality, both the Python command you'll type and where you'll type it vary per platform. On Windows, macOS, and Android, for example, `py`, `python3`, and `python`, respectively, do the job as follows:

```
$ py Python 3.12.3 (...etc..., Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> ^Z $ python3 Python 3.12.2 (...etc..., Feb 6 2024, 17:02:06) [Clang 13.0.0...] on darwin Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> ^D $ python Python 3.11.4 (...etc..., Jul 2 2023, 11:17:00) [Clang 14.0.7...] on linux Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> ^D A python3 works on Android and Linux, too, if you want to narrow this to one exception—for Windows' py. This book uses python3 in platform-neutral
```

examples just because it works almost everywhere. If you're working on Windows, instead use `py` (or `py -3` to ensure Python 3.X); `python3` works on Windows, too, but is reserved for a lesser Store install today per Appendix A. Also per that appendix, all these commands must be on your system search path (generally known as `PATH`) and require a full pathname if they're not, but most installs set this up automatically. Typing a Python command at your system prompt like this begins an interactive Python session (i.e., REPL). The `$` character at the start of listings here stands for a generic system prompt on all platforms throughout this book—it will likely vary on your device and is not input that you type yourself. On Windows, a `Ctrl+Z` key combo (followed by `Enter`) ends this session as shown; on Unix (which includes macOS, Linux, and Android), it's `Ctrl+D` instead. You'll notice that the interactive prompt opens with a message that identifies the Python being used (e.g., "3.12.3" is Python 3.12) and the platform it's running on (e.g., "win32" for Windows, "darwin" for macOS, and "linux" for both Linux and Android in Python 3.12), followed by a line with tips for more info. We're going to omit these opening lines in this book, except where they're helpful. The notion of a system prompt (a.k.a. shell or console) where you type the Python command is generic, but exactly how you access it varies by platform. It might be Command Prompt or PowerShell on Windows; Terminal o

n macOS and Linux; the Termux app on Android; or a dedicated terminal screen in some apps like Pydroid 3. On most platforms, you can also start an interactive session in ways that don't require typing a command at all, but they vary even more widely. On Windows, for example, a Start-menu option opens a similar REPL, as do the IDLE GUI on all PCs, dedicated screens in apps on Android and iOS, and some web-browser interfaces. We'll cover some of these options ahead, but see Appendix A for more platform tips, and the web for more help with other options. Anywhere you see the `>>>` prompt, though, you're in an interactive Python session and REPL—you can type any Python statement or expression here and run it immediately by simply pressing the Enter key (or similar button). We will do so in a moment, but first we need to get a few admin details sorted out to make sure all readers are set to go.

中文翻译

从多数标准衡量，运行 Python 代码最简单的方式，就是在 Python 的交互式命令行里输入代码。它有时被称为交互式提示符 (interactive prompt)，常常更简洁地称作 REPL (即 read-eval-print loop, "读取—求值—打印"循环)。启动这个命令行有多种途径——在编码 GUI 或 App 里、在网页笔记本里、在系统控制台里，等等。假设 Python 已经作为可执行程序安装在你设备上，那么启动交互式解释器会话最具平台无关性的方式，就是直接在设备控制台提示

符下、不带任何参数地输入一个 Python 命令。由于这种方式最常见、几乎通用、也最简单，我们就从这里开始。尽管这个方案具有通用性，你要输入的 Python 命令以及在哪里输入，都是随平台而异的。例如在 Windows、macOS 和 Android 上，分别用 `py`、`python3` 和 `python` 就能做到，如下所示：

```
$ py Python 3.12.3 (...等等..., 2024 年 4 月 9 日 14:05:25) [MSC v.1938 64 位 (AMD64)] on win32
输入 "help"、"copyright"、"credits" 或 "license" 获取更多信息。>>> ^Z $ python3 Python 3.12.2 (...等等..., 2024 年 2 月 6 日 17:02:06) [Clang 13.0.0...] on darwin
输入 "help"、"copyright"、"credits" 或 "license" 获取更多信息。>>> ^D $ python Python 3.11.4 (...等等..., 2023 年 7 月 2 日 11:17:00) [Clang 14.0.7...] on linux
输入 "help"、"copyright"、"credits" 或 "license" 获取更多信息。>>> ^D
```

如果你想把这个差异收窄到唯一一个例外（即 Windows 的 `py`），那么 `python3` 在 Android 和 Linux 上也都能用。本书在平台无关的示例中使用 `python3`，仅仅因为它几乎到处都能用。如果你在 Windows 上工作，请改用 `py`（或用 `py -3` 来确保是 Python 3.X）；`python3` 在 Windows 上也能用，但按附录 A，如今它被保留给功能较弱的 Store 安装。同样依据该附录，所有这些命令都必须位于系统搜索路径（通常称为 PATH）中；若不在，就需要给出完整的路径名，不过大多数安装都会自动设置好。像这样在系统提示符下输入一个 Python 命令，就开启了一个交互式 Python 会话（即 REPL）。本书列出的每条命令开头的 `$` 字符，在所有平台上都表示通用的系统提示符——它在你的设备上可能不同，而且它不是需要你自己输入的。在 Windows 上，按 `Ctrl+Z`（之后回车）如所示

那样结束会话；在 Unix（包括 macOS、Linux 和 Android）上则是 Ctrl+D。你会注意到交互式提示符打开时会先打印一段信息，用来标明所用的 Python 版本（如 "3.12.3" 即 Python 3.12）和运行平台（如 Windows 是 "win32"、macOS 是 "darwin"、"linux" 对应 Linux 和 Android 的 Python 3.12），随后是一行获取更多信息的提示。本书除了必要时会省略这些开头几行。系统提示符（又名 shell 或 console）的概念是通用的——你在那里输入 Python 命令，但具体如何访问它则因平台而异。在 Windows 上可能是命令提示符（Command Prompt）或 PowerShell；macOS 和 Linux 上是 Terminal；Android 上是 Termux 应用；或者像 Pydroid 3 这类应用中专门的终端屏幕。在大多数平台上，你还可以用完全不需输入命令的方式启动交互式会话，但那些方式差异更大。例如在 Windows 上，开始菜单里有一个选项会打开类似的 REPL；所有 PC 上的 IDLE GUI、Android 和 iOS 应用中的专用屏幕、以及某些网页浏览器接口也都能做到。我们稍后会覆盖其中一些选项，但更多平台技巧请看附录 A，其它选项可上网查找更详细的帮助。不过，任何你看到 >>> 提示符的地方，你就正处于交互式 Python 会话（REPL）中——你可以在那里输入任意 Python 语句或表达式，只要按下回车键（或类似的按键）就能立即运行。我们马上就会这样做，但首先还需要处理几个“管理细节”，确保所有读者都准备就绪。

深度理解

·核心概念：REPL = Read（读取）– Eval（求值）– Print（打印）– Loop（循环）。这个名字精确描述了交互式会话的工作循环：读一行输入 → 编译并执行 → 打印结果

→ 等待下一行。

·命令名差异的来龙去脉：

· py: Windows 的 launcher (启动器)，专为 Windows 设计，可以让你在同一机器上选择不同 Python 版本 (py -3)。微软商店 (Store) 安装的 python3 是个入口别名，能力受限，所以本书主推 py。

· python3: Unix 世界的标准命名，明确区分 Python 3。

· python: 部分发行版把 Python 2 和 3 分别叫 python/python3，但如今 (Python 2 已 EOL) 多数系统 python 也指向 3。

· PATH 是什么：系统“搜索路径”。当你输入 py，系统按 PATH 中列出的目录逐个查找名为 py.exe 的程序。若 Python 安装目录不在 PATH 里，就要输完整路径 (如 C:\Python312\python.exe)。

·退出方法：Ctrl+D (Unix, 发送 EOF) vs Ctrl+Z (Windows, 传统 EOF)。这是历史遗留——Windows 的 EOF 用 ^Z。

·启动横幅信息：第一行告诉你三件事——版本号、编译环境 (如 MSC = 微软编译器)、平台 (win32/darwin/linux)。这些是排错时的重要线索。

·生活类比：REPL 就像计算器加记事本——你按一个数字计算器立刻给结果，但算完就忘。存盘要靠“脚本文件” (后面讲)。

·初学者误区：①把 \$、>>> 也敲进去了 (下一节专门讲

) ; ②以为 REPL 里写的代码能被保存——不能；③分不清"系统提示符"和"Python 提示符"——\$ 或 PS> 是操作系统，>>> 是 Python，它们是两套不同的环境。

代码分析

```
# ""REPL Python
py          # Windows Python 3.xlauncher
python3     # macOS/Linux/Android Python 3
python      # Linux

#
# Windows Ctrl+Z
# Unix/macOS/Linux/Android Ctrl+D
```

处理流程：

- 当你在系统控制台输入 `py` 并回车，操作系统根据 `PATH` 找到解释器可执行文件，把它作为一个新进程启动。
- Python 进程启动后进入 REPL：打印版本横幅 → 打印 `>>>` 提示符 → `read` 等待你的输入。
- 你在 `>>>` 后输入任意语句，Python 内部会做：源码 → 词法分析 → 编译为字节码 (bytecode) → 交给 PVM (Python 虚拟机) 执行 → 若有结果则 `print` 到屏幕。这正是第 2 章讲过的执行模型，在这里第一次有了直观体验。
- 你按 `Ctrl+D`/`Ctrl+Z` 时，进程收到 EOF 信号，主循环结束，解释器退出，控制权还给系统 shell。

3.3.2 在哪里运行：代码文件夹 (Where to Run: Code Folders)

英文原文

Now that you're starting to learn how to run code, you'll also need to know where to run code. You can save and run code anywhere you can make files, but this book has two recommendations for using its examples, especially for newcomers: Work in a dedicated code folder. To avoid stomping on other content, run this book's examples from a dedicated code folder (a.k.a. directory) on your device. For instance, this book runs all its code in a folder nested in the user account (or "home") folder on each device. Your code folder can be located wherever you like and called whatever you wish, but running out of one folder will help you keep track of your work and simplify some tasks. Work in per-chapter subfolders of your code folder. To avoid clutter and filename collisions, also organize your code into per-chapter subfolders, nested in your dedicated code folder. Per-chapter subfolders will ensure that imports in examples will work without advanced settings (that you'll learn later). There's more on imports and directories in an upcoming note. For now, keep in mind that console commands in this book will implicitly be run in a per-chapter subfolder of a dedicated code folder on the host. The folder name will normally be omitted for platform neutrality and is irrelevant to the code. If you'll be using this book's examples package, it's already done the setup work for you.

For example, unzipping the examples creates a code folder named LP6E, whose Chapter03 subfolder has this chapter's code. If you'll be using this package to avoid typing code or copying from eMedia, simply run its examples in the subfolder of the chapter you're studying. This is where REPLs will be started and where script files will be run. As you'll learn later, this is also where data files that our scripts create will show up, unless scripts use filenames with explicit folder paths. If you'll be creating code on your own, you should probably create a similar code directory structure of your own before we move on. On PCs, use your system's file explorer, or run a command line: `mkdir folder` works on Windows and Unix. On Android, you can also use a file explorer (or `mkdir` in Termux), but be sure to pick a folder accessible to your Python app. Some coding interfaces may offer other ways to create folders; see your tool for info. Once your per-chapter code folders are set to go, always start there to write, save, and run the examples in this book. How you'll do this depends on your usage mode. In consoles, a portable `cd` folder command changes directories. In a GUI like IDLE, opening and running a file may go to its folder. And in other interfaces, you might launch a REPL for a file in the UI or use other schemes too varied to cover here. As a fallback, Python's `os.getcwd()` you'll meet shortly shows the current directory, and its `os.chdir('path')` changes it—as long as you import `os` first.

中文翻译

既然你开始学习如何 (how) 运行代码, 你也需要知道在哪里 (where) 运行代码。你可以在任何能创建文件的地方保存和运行代码, 但本书对使用其示例有两个建议, 尤其对新手如此: 在一个专门的代码文件夹中工作。为避免弄乱其它内容, 请从设备上一个专用的代码文件夹 (folder, 又名目录 directory) 中运行本书的示例。例如, 本书在所有设备上都在用户账户 (或"主目录"home) 文件夹下的一个子文件夹里运行全部代码。你的代码文件夹可以放在任何你喜欢的地方、叫任何你喜欢的名字, 但总是从一个文件夹出发运行, 能帮你追踪工作成果, 并简化一些任务。在代码文件夹下按章节划分子文件夹。为避免杂乱和文件名冲突, 还要把你的代码组织到专用的代码文件夹之下、按章节划分的子文件夹 (subfolders) 里。按章节划分子文件夹能确保示例中的 imports (导入) 无需高级设置 (你以后会学到) 就能正常工作。关于导入和目录, 后面的提示中还有更多说明。眼下只需记住: 本书中的控制台命令, 隐式地都是在宿主机上 "专用代码文件夹下的一个按章节划分的子文件夹" 中运行的。为了平台无关性, 文件夹名通常会被省略, 而且它与代码本身无关。如果你要使用本书的示例包 (examples package), 它已经替你完成了这套准备工作。例如, 解压示例包会创建一个名为 LP6E 的代码文件夹, 其中的 Chapter03 子文件夹就放着本章的代码。如果你想用这个包来避免手动输入代码或从电子资源复制, 那就在你正在学习那章的对应子文件夹里运行示例。这就是 REPL 启动、脚本文件运行的地方。正如你以后会学到的, 脚本创建出来的数据文件 (data files) 也会出现在这里——除非脚本使用了带显式路

径的文件名。如果你打算自己创建代码，那你很可能应该在继续之前先创建一个类似的目录结构。在 PC 上，用系统的文件资源管理器，或运行命令行：`mkdir folder` 在 Windows 和 Unix 上都可用。在 Android 上，也可以用文件资源管理器（或在 Termux 里用 `mkdir`），但务必选一个你的 Python 应用能访问的文件夹。某些编码接口可能提供其它创建文件夹的方式；请查阅你的工具说明。一旦你的按章节代码文件夹就绪，就总是从那出发来编写、保存和运行本书的示例。具体怎么做取决于你的使用模式。在控制台中，可移植的 `cd folder` 命令可以切换目录。在 IDLE 这类 GUI 中，打开并运行文件往往就会定位到它的文件夹。而在其它接口中，你可能通过界面为某个文件启动一个 REPL，或使用其它这里不便尽述的方式。作为兜底手段，你马上就会见到的 Python `os.getcwd()` 能显示当前目录，`os.chdir('path')` 能改变它——只要你先 `import os`。

深度理解

- 核心概念：当前工作目录（current working directory，CWD）是理解 Python 文件运行的关键。你"在哪里 `cd`"，决定了你 `import` 能找到什么、脚本把数据文件写到哪。
- 为什么按章节分子文件夹：①避免文件名冲突（每个章节练习都爱起 `script1.py`）；②让 `import` 以最朴素的方式生效——因为 `import` 默认会搜索当前目录，只要你的代码都在一起，就无需配置 `PYTHONPATH`。
- `os.getcwd()/os.chdir()` 的现实用途：脚本不知道自己"在哪个目录被运行"，这两个函数让程序在运行时感知并改

变工作目录——这是自动化脚本（批量处理文件）的常规操作。

·生活类比：工作目录就像你"站在房子的哪个房间"。你在厨房切菜（运行脚本），菜刀（模块）就必须在你伸手够得到的地方（当前目录 + 搜索路径）。

·初学者误区：把脚本放在一个地方、却在另一个目录运行，导致 import 报 `ModuleNotFoundError`，或数据文件"凭空消失"（其实写到了运行时的目录里）。在哪个目录启动 Python，脚本就在那个目录里找文件、写文件。

3.3.3 不要输入什么：提示符与注释 (What Not to Type: Prompts and Comments)

英文原文

Speaking of commands, remember not to type the `$` character used at the start of this book's command lines to denote a system console prompt; type just the text after these prompts. This may sound simple to experienced programmers, but it's a very common first error for beginners, and we're not excluding anyone here. Similarly, do not type the `>>>` and `...` prompt characters shown at the start of lines in interpreter interaction listings and used by this book to denote code run in a REPL; type just the text after these prompts. These are prompts that Python's standard REPL displays automatically as visual guides for interactive code entry and may or may not appear in your interface.

For instance, the ... prompt is used for continuation lines in some REPLs, but is just a label in IDLE, and is omitted by some of this book's listings for easier copy/paste; either way, don't type it yourself. To help you remember this, user inputs you must type are shown in bold in this book, and prompts are not. Also keep in mind that commands typed after these system and Python prompts are meant to be run immediately and are not generally intended to be saved in the source files we will be creating; you'll see why this distinction matters ahead. In the same vein, you normally don't need to type text that starts with a # character in this book's code listings—as you'll learn later, these are comments, not executable code. Except when # is used to introduce a #! directive at the top of a script, you can safely omit it and the rest of the line that follows it (there's more about #! in Appendix A).

中文翻译

说到命令，请记住不要输入 \$ 字符——它被用在本书命令行开头，表示系统控制台提示符；只输入这些提示符之后的文本即可。对有经验的程序员来说这听起来很简单，但这是初学者非常常见的第一个错误，我们这里可没有排除任何人。同样地，也不要输入解释器交互列表里行首显示的 >>> 和 ... 提示符字符——本书用它们表示在 REPL 中运行的代码；只输入这些提示符之后的文本。这些提示符是 Python 标准 REPL 自动显示、作为交互式代码录入的可视化向导的，你的界面里可能出现也可能不出现。例如，... 提示符在某

些 REPL 中用于续行 (continuation lines)，但在 IDLE 里只是标签，而且本书某些列表为了方便复制粘贴会把它省掉；无论如何，都不要自己输入它。为了帮你记住这一点，本书中必须由你输入的用户输入以粗体显示，而提示符不是粗体。同时要记住：在系统提示符和 Python 提示符之后输入的命令，是要立即运行的，一般不打算保存到我们将要创建的源文件中；你稍后就会明白这个区别为什么重要。同理，在本书的代码列表里，通常你也不需要输入以 # 字符开头的文本——正如你以后会学到的，这些是注释 (comments)，不是可执行代码。除非 # 被用来引入脚本顶部的 #! 指令，否则你可以放心地省略它及该行中其后剩余的部分（关于 #! 的更多内容见附录 A）。

深度理解

- 核心概念：区分三类“你看到的符号”——系统提示符 (\$、PS>)、Python 提示符 (>>>、...)、注释标记 (#)。三者都不是你要输入的执行内容。
- 两个不同的解释器层级：\$ 后的命令交给系统 shell (command.com/bash)，>>> 后的命令交给 Python REPL。把 python3 script1.py 输进 Python 的 >>> 会报语法错误——因为 python3 是 shell 的命令、不是 Python 语句。同理，把 >>> 输进文件会语法错误。
- # 注释：从 # 到行尾的任何文本，Python 一律忽略。它为人类而存在，不参与执行。
- #! (shebang) 是个例外：脚本第一行的 #!/usr/bin/env python3 是给操作系统看的——当你在 Unix 上直接 ./script.py 运行时，系统靠它决定用哪个解释器。

·生活类比：教科书上的选择题答案写在括号里，你做题时不该把括号抄进去。\$ />>>/# 就是那些"括号"。

·初学者误区：把整段带提示符的代码直接复制进文件 → 语法错误！这是新手最容易、也最典型的翻车现场。

3.3.4 其它 Python REPL (Other Python REPLs)

英文原文

Having said all that, you should also know that Python REPLs can also be had in systems that convolute the traditional model covered in this chapter. IPython, for example, provides an alternative, separately installed, and enhanced Python interactive session, which labels commands by number and doesn't use >>> prompts. Jupyter notebooks provide the IPython REPL, too, and run it in a web browser instead of system console (we'll explore Jupyter later in this chapter). To muddy this story further, the PyPy system of Chapter 2 uses >>>> for its REPL prompts to distinguish it from CPython's ternary >>> (though you may have to look hard to tell); the IDLE GUI covered ahead displays >>> off to the side (and not in a system console); smartphone apps may vary too (see Appendix A); and the interactive prompt can technically be changed to anything (it's `sys.ps1` to your code, and can be set in a startup file, per Python docs). All of which means that your Python interactive session may differ from the

e mainstream model that this chapter often employs. Depending on your tools and goals, you might see a different prompt or none at all, and you might type interactive Python code into a web browser, GUI, or a `pp` instead of the system console. In general, though, this book recommends the traditional and simpler options it demos and covers, when you're first starting out. IPython and Jupyter, for example, have learning curves of their own, and Jupyter is geared toward scientific work, which is just one of many Python roles. If and when you opt to use alternative coding interfaces like these, and others sure to arise in the future, extrapolate to their REPLs' minor differences as needed.

> NOTE. REPL futurism: The standard CPython is scheduled to improve its interactive REPL interface in version 3.13, not yet released as this book was written. Per plans, the REPL will gain color prompts, automatic indentation, multiline editing and paste, and colorized exceptions borrowed from other UIs and IDEs. Colors won't matter in this book's print version, of course, and the future is not yet written, but a more colorful future may have arrived by the time you read this note.

> > Tip: Setting environment variable `PYTHONCOLORS` to 0 disables the new REPL colors if you find that future distracting.

中文翻译

说了这么多，你还应该知道：Python REPL 也可以存在于那些把本章所述的传统模型"搅浑"的系统里。例如 IPython 提供另一种独立安装、功能增强的 Python 交互式会话，它用编号标记命令，不使用 `>>>` 提示符。Jupyter 笔记本同样提供 IPython REPL，并在网页浏览器里运行它，而不是在系统控制台里（我们本章稍后会探讨 Jupyter）。把情况搅得更浑的是，第 2 章的 PyPy 系统用 `>>>>` 作为它的 REPL 提示符，以区别于 CPython 的三字符 `>>>`（尽管你可能得使劲看才分得清）；后面讲到的 IDLE GUI 把 `>>>` 显示在侧边（而不是系统控制台里）；智能手机应用也可能各异（见附录 A）；而且交互式提示符技术上可以被改成任何东西（对你的代码而言它是 `sys.ps1`，可按 Python 文档在启动文件里设置）。这一切都意味着你的 Python 交互式会话可能与本章常用"主流模型"有所不同。取决于你的工具和目标，你看到的提示符可能不同、或者根本没有，而你可能是在网页浏览器、GUI 或 App 里而不是在系统控制台里输入交互式 Python 代码。不过总体而言，在最初起步时，本书推荐它演示和覆盖的传统、简单选项。例如 IPython 和 Jupyter 都有自己的学习曲线，而且 Jupyter 面向科学计算工作——那只是 Python 众多角色之一。如果你选择使用这类以及将来必然出现的其它替代编码接口，就按需类推它们的 REPL 的一些细微差别即可。> 注意。REPL 未来主义：标准 CPython 计划在 3.13 版本改进其交互式 REPL 界面（本书写作时尚未发布）。按计划，REPL 将获得彩色提示符、自动缩进、多行编辑与粘贴、以及借鉴自其它 UI 和 IDE 的彩色化异常。当然，颜色对本书印刷版无关紧要，而且未来尚未定型——但等你读到这条注记时，也许一个更多彩的未来已经到来了。> > 提示：如果你觉得那个未

来让人分心，把环境变量 PYTHONCOLORS 设为 0 就能关闭新的 REPL 颜色。

深度理解

·核心概念：本书示范的是 CPython 标准 REPL，但 REPL 世界是多元的——IPython/Jupyter（增强版、笔记本式）、PyPy（提示符都不同）、IDLE（GUI 内嵌）、各种 App。

·sys.ps1 是提示符的本体：>>> 不是魔法字符，它是启动文件里默认可改的字符串。Python 里 sys.ps1 就是主提示符，sys.ps2 是续行提示符 ...。你可以改成 sys.ps1 = '>>>' 之外的任意值——这说明 Python 的"界面"也是可编程对象。

·为什么作者坚持"传统简单方案"：教学场景下，用 IPython/Jupyter 等于"套娃"——它们之上的交互层（编号、魔法命令、笔记本模型）会给初学者额外认知负担。先把标准 REPL 玩熟，再决定要不要进阶工具。

·台后知识：3.13 的 REPL 改进（PYTHONCOLORS）提醒我们：Python 的交互界面本身也是持续进化的产品。这不算"编造未来"，这是书里明说的规划。

·生活类比：都是"打电话"，标准 REPL 就像老式座机——直白、通用；IPython/Jupyter 像视频会议软件——功能多但要求你得先会装会用。

·初学者误区：①听说 IPython 好用就去装，结果不会用魔法命令反而困惑；②在 PyPy 里看到 >>>> 以为自己环境坏了——那是正常设计。

3.4 交互式运行代码 (Running Code Interactively)

英文原文

With those preliminaries out of the way, let's move on to typing some actual code. However it's started, the Python interactive session begins by printing some informational text (again, mostly omitted hereafter), then prompts for input with `>>>` (or similar) when it's waiting for you to type a new Python statement or expression. When working interactively, the results of your code are displayed below the `>>>` input lines after you press the Enter key (or similar). For instance, here are the results of two Python print statements:

```
$ python3 >>> print('Hello world!') Hello world!
>>> print(2 * 8) 256
```

There it is—we've just run some Python code (it's not much, but it proves the point). We're still skipping most code details for now, but in brief, `print` is a built-in tool that sends a line displaying whatever you pass to it, to wherever you're working; because it's a function call in Python, the parentheses in this code are required. We've used it here to display a Python string and an integer, as shown by the output lines that appear after each `>>>` input line. When coding interac

tively like this, you can type as many Python commands as you like; each is run immediately after it's entered. Moreover, because the interactive session automatically prints the results of expressions you type, you don't usually need to say "print" explicitly at this prompt; the format of the following automatic displays can differ slightly from print, but it's not yet important to know how:

```
>>> language = 'Python'>>> language'Python'  
>>> 2 * 8 * 256  
>>> ^Z # Use Ctrl-D (on Unix) or Ctrl-Z (on Windows  
) to exit $
```

Here, the first line saves a value by assigning it to a variable (language), which is created by the = assignment; and the last two lines typed are expressions (language and 2 * 8), whose results are displayed automatically. Again, to exit an interactive session like this and return to your system prompt, type Ctrl+D on Unix-like machines, and Ctrl+Z on Windows. In the IDLE GUI discussed later, either type Ctrl+D everywhere, or simply close the window. Notice the italicized note about this on the right side of this listing—starting with # here. This book uses these throughout to add remarks about what is being illustrated, but you don't need to type this text yourself; if you do, they're ignored by Python as comments. In fact, much like system \$ and Python >>> prompts, you shouldn't type this when

it's on a system command line; the `#...` part is ignored by Python but may be an error in system shells. Now, we didn't do much in this session's code—just typed some Python print and assignment statements, along with a few expressions, all of which we'll study in detail later. The main thing to notice is that the Python REPL executes the code entered on each line immediately, when the Enter key (or similar) is pressed. For example, when we typed the first print statement at the `>>>` prompt, the output (a Python string) was echoed back right away. There was no need to create a source code file, and no need to run the code through a compiler and linker first, as you'd normally do when using a language such as C or C++. Strictly speaking, interactive code is compiled to bytecode in memory and run by the PVM in CPython (see Chapter 2), but you don't need to care. As you'll see in later chapters, you can also run multiline statements (e.g., for loops) at the interactive prompt; such a statement may prompt for continuation lines with `...` as noted earlier and runs immediately after you've entered all of its lines and pressed Enter twice to add a blank line. Blank lines aren't required (and are ignored) in code files, but are needed to let some REPLs know your statements are complete. Also, bear in mind that the current and standard REPL runs just one statement at a time—don't paste large code blocks at its prompt!

中文翻译

排除了那些预备事项，让我们继续输入一些真正的代码。无论以何种方式启动，Python 交互式会话都会先打印一些信息文本（以后大多省略），然后在等待你输入新的 Python 语句或表达式时，用 `>>>`（或类似符号）提示你输入。在交互式工作时，按下回车键（或类似按键）后，你的代码结果会显示在 `>>>` 输入行的下方。例如，下面是两条 Python `print` 语句的结果：`$ python3 >>> print('Hello world!')` Hello world! `>>> print(2 * 8)` 256 就是这样——我们刚刚运行了一些 Python 代码（虽然不多，但证明了这一点）。目前我们仍略过大部分代码细节，但简要地说，`print` 是一个内建工具，它把你传给它的任何内容以一行文本的形式发送到你正在工作的任何地方；因为它是 Python 中的函数调用（function call），这段代码里的括号是必须的。我们在这里用它显示了一个 Python 字符串和一个整数，正如每条 `>>>` 输入行下方出现的输出行所示。像这样交互式编码时，你想输入多少条 Python 命令都行；每条命令在输入后立即运行。此外，因为交互式会话会自动打印你输入的表达式的结果，所以你通常不必在这个提示符下显式地说 `"print"`；下面的自动显示格式可能与 `print` 略有不同，但现在不必关心：`>>> language = 'Python'>>> language` Python `>>> 2 * 8` 256 `>>> ^Z` # 在 Unix 上用 Ctrl-D、在 Windows 上用 Ctrl-Z 退出\$ 这里，第一行通过把值赋值给一个变量（variable）（`language`）来保存它，这个变量由 `=` 赋值创建；最后输入的两行是表达式（expressions）（`language` 和 `2 * 8`），其结果被自动显示出来。同样，要退出这样的交互式会话回到系统提示符，在类 Unix 机器上

按 Ctrl+D，在 Windows 上按 Ctrl+Z。在后面讲到的 IDLE GUI 中，要么任何地方都按 Ctrl+D，要么直接关闭窗口。请注意这个列表右侧的斜体注释——从这里以 # 开头。本书全篇都用它们来补充说明这段列举在演示什么，但你不必自己输入这段文字；就算你输入了，它们也会被 Python 当作注释（comments）忽略。事实上，和系统 \$、Python >>> 提示符很类似，当它出现在系统命令行上时你也不该输入它；#...部分会被 Python 忽略，但在系统 shell 里可能是错误。好吧，本次会话的代码没做多少事——只是输入了一些 Python print 和赋值语句，外加几个表达式，这些我们以后都会详细学习。最主要观察点是：Python REPL 在按下回车键（或类似按键）时，会立即（immediately）执行每行输入的代码。例如，当我们在 >>> 提示符下输入第一个 print 语句时，输出（一个 Python 字符串）立刻被回显出来。无需创建源代码文件，也无需像使用 C 或 C++ 这类语言那样，先让代码过一遍编译器和链接器。严格来说，交互式代码是在内存中被编译成字节码、再由 CPython 中的 PVM 运行的（见第 2 章），但你不需要关心这些。正如你在后面的章节会看到的，你也可以在交互式提示符下运行多行语句（例如 for 循环）；这样的语句可能像前面提到的那样，用 ... 提示你输入续行，并且在你输完所有行、连续按两次回车补充一个空行后立即运行。在代码文件里空行不是必需的（并且会被忽略），但在某些 REPL 里，要让它们知道你的语句已经完整，空行是必要的。另外请记住：目前的标准 REPL 一次只运行一条语句——不要往它的提示符里粘贴大段代码块！

深度理解

- 核心概念：交互模式的"立即执行"特性，源自它没有"编译-链接"阶段。你每按一次回车，这一行的源码就被实时编译执行。这让 Python 成为"所见即所得"的语言——而 C/C++ 需要先编译整个程序再运行。
- 底层细节（回应第 2 章）：严格说，交互式代码也要先编译成字节码——只是在内存里进行，不落盘为 .pyc 文件。所以它依然是"编译为字节码 → PVM 执行"，只是省掉了文件往返。
- 表达式自动回显 vs print 的区别：在 REPL 里，language 和 2.8 这种表达式的值会被 REPL 自动"打印"，无需 print。而 print(...) 是语句调用，把值送往标准输出流。两者的输出格式略有差异（表达式回显常带 repr() 的效果，字符串会带引号）。这是 REPL 特有的行为——脚本文件里没有自动回显，文件里必须显式 print。
- "按两次回车"的机制：多行语句（如 for）需要用户连按两次回车——第一次结束复合语句的某个子句、第二次的空行表示整个语句块结束。空行是 REPL 判断"语句输入完毕"的信号。这是解释型语言的交互特性。
- 一次一条语句：标准 REPL 一次只执行一条逻辑语句，粘贴大块代码会造成麻烦——这是它在交互模式下的天然限制。
- 生活类比：表达式自动回显就像自动回复的语音助手——你说一句它答一句；而 print 是你明确要求它记在便签上，去向明确。

·初学者误区：①以为自己敲的 `language = 'Python'` 会"保存"成文件——不会，交互会话结束一切归零；②在脚本文件里期望表达式自动输出——文件里没有自动回显，不 `print` 就什么都看不见。

代码分析

```
print('Hello world!') #  
print(2 ** 8)          # 2 8 256  
language = 'Python'   # language  
language               # REPL 'Python'  
2 ** 8                 # REPL 256
```

解释器如何处理：

- `print('Hello world!')` 是函数调用。Python 先在当前命名空间查找名字 `print`（它是个内建函数 `builtin`），然后把字符串对象 `'Hello world!'` 作为参数传入，函数把该字符串写到标准输出（`stdout`），随后换行。这里的括号是必需的——`print` 在 Python 3 中是函数（不是像 Python 2 那样的语句）。
- `print(2 8)`：先对 `2 8` 求值。是幂运算符，CPython 把它编译为 `BINARYOP`（幂）字节码，在整数对象上调用乘幂算法，得到整数对象 `256`，再传给 `print` 输出。
- `language = 'Python'`：赋值语句。右侧的字符串字面量在内存中创建字符串对象，左侧的 `language` 只是指向该对象的名字（`name binding`，名字绑定）。
- 单独一行 `language`：这是一个表达式。REPL 会对它求值得到字符串对象，然后自动调用其显示表示（`repr`）并打印——所以能看到带单引号的 `'Python'`。在脚本文件中

这行不会有任何输出。

· 2 8: 同理会话自动显示 256, 但注意和 `print(2 8)` 显示的 256 在本文末尾其实相同, 只是层次不同: 一个是表达式自动回显, 一个是函数显式输出。

3.5 为什么用交互式提示符? (Why the Interactive Prompt?)

英文原文

The interactive prompt runs code and echoes results as you go, but it doesn't save your code in a file. Although this means you won't do the bulk of your real-world coding in interactive sessions, the interactive prompt turns out to be a great place to both learn the language and test program files on the fly. Here's a quick rundown of these roles.

中文翻译

交互式提示符一边运行代码一边回显结果, 但它不会把你的代码保存到文件里。尽管这意味着你不会在交互式会话中完成大部分实际编码工作, 但交互式提示符实际上是学习语言和即时测试程序文件的绝佳场所。下面快速盘点这些角色。

深度理解

· 核心概念: 交互模式的定位是学习 + 测试的临时沙盒, 不是生产环境。作者给出全书第一条“经验法则”——对代

码有疑问就开 REPL 试。

·为什么"不会崩": 你不小心敲错代码, Python 只会抛出异常并打印 Traceback (回溯信息), 解释器进程本身不会崩溃。这在 C 里意味着段错误可能直接让进程崩溃。

·生活类比: 教练在泳池边先让你在浅水区 (REPL) 扑腾熟悉水性, 再让你下水道 (脚本文件) 正式游。

代码分析

```
>>> 'Hack!' * 8          #  
'Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!'  
>>> 2 ** 500             # Python  
>>> 1 / 0                # ZeroDivisionError
```

·'Hack!' 8: 字符串与整数相乘 = 字符串重复拼接。运算符在数字上是乘法、在字符串 (和列表等序列) 上是重复。字节码层面, CPython 会为 str int 走序列重复实现, 直接分配拼接后的新字符串。

·"实验破坏不了任何东西"的保证: 纯 Python 表达式的求值是"沙盒式"的——它只会操作内存中的对象, 不碰文件系统、不碰操作系统。真正危险的操作 (删文件、执行 shell 命令) 只能通过显式 import os / import shutil 后调用其函数才能做到。

3.5.1 学习 (Learning)

英文原文

Because it executes code immediately, the interactive prompt is ideal for experimenting with code and lear

ning the language, and we'll be using it throughout this book to demonstrate smaller examples and amplify concepts. In fact, this is the first rule of thumb to remember: if you're ever in doubt about how a piece of Python code works, fire up the interactive command line and try the code out to see what happens. For instance, suppose you're reading a Python program's code and you come across an expression like `'Hack!' * 8` whose meaning you don't understand. At this point, you can spend 10 minutes wading through manuals, books, and the web to try to figure out what the code does, or you can simply run it interactively:

```
$ python3 >>>'Hack!' * 8 # Learning by trying'Hack!Hack!Hack!Hack!Hack!Hack!Hack!'
```

This immediate feedback you receive at the interactive prompt is often the quickest way to deduce what a piece of code does. Here, it's clear that it does string repetition: in Python, `*` means multiply for numbers, but repeat for strings—it's like concatenating a string to itself repeatedly (there's more on strings in Chapter 4). Chances are good that you won't break anything by experimenting this way—at least, not yet. To do real damage, like deleting files and running shell commands, you must really try by importing modules explicitly (we'll sample tools that can make you that dangerous later in this chapter). Straight Python code, though, is almost always safe to run. For instance, watch what happens when you make a mistake at the in

teractive prompt:

```
>>> X # Making mistakes is OK
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'X' is not defined
```

In Python, using a variable before it has been assigned a value is always an error—otherwise, if names were filled in with defaults, some errors might go undetected until it's too late. This means you must initialize counters to zero before you can add to them, must initialize lists to empty before extending them, and so on. You don't need to declare variables in Python, but they must be assigned before you can fetch their values. Other error messages try to be more helpful in Python today with "Did you...?" tips—as when looking for the `sys.ps1` prompt hook mentioned earlier. You must import modules like `sys` before using them (though you probably won't need the reminder fairly soon, and we'll be truncating some error messages in this book for brevity):

```
>>> sys.ps1 # Requires "import sys"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'sys' is not defined. Did you forget to
import 'sys'?
```

You'll learn more about all that later. The important point here is that you won't crash Python or your computer when you make a mistake this way. Instead, you get a meaningful error message pointing out the

mistake and the line of code that made it, and you can continue on in your session or script. In fact, once you get comfortable with Python, its error messages may often provide as much debugging support as you'll need (watch for more about debugging options in the sidebar "Debugging Python Code").

中文翻译

因为它能立即执行代码，交互式提示符非常适合实验代码和学习语言，本书全篇都会用它演示较小的示例、放大概念。事实上，这是要记住的第一条经验法则：如果你对某段 Python 代码如何工作有任何疑问，就启动交互式命令行，把代码实际跑一遍看看会发生什么。例如，假设你在读一个 Python 程序，遇到一个你不理解含义的表达式，比如 'Hack!' * 8。此时，你可以花 10 分钟翻手册、查书、上网搞清这段代码做什么，或者干脆交互式地运行它：

```
$ python3 >>>'Hack!' * 8
```

 # 通过尝试来学习 'Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!' 你在交互式提示符下获得的即时反馈，常常是推断一段代码做什么的最快捷方式。这里很清楚：它在做字符串重复。在 Python 中，`*` 对数字是乘法、对字符串是重复——就像把字符串反复拼接到自己身上（第 4 章会详谈字符串）。以这种方式实验，你很可能不会破坏任何东西——至少现在不会。要造成真正的破坏，比如删除文件、执行 shell 命令，你必须认认真真地通过显式导入模块（`importing modules explicitly`）才行（本章稍后会示范那些能让你“变危险”的工具）。但纯粹的 Python 代码几乎总是可以安全运行的。例如，看看你在交互式提示符犯错过会发生什么：

```
>>> X
```

 # 犯错过没关系 `Traceback (most re`

cent call last): File "<stdin>", line 1, in <module> NameError: name 'X' is not defined 在 Python 中，在变量被赋值前就使用它永远是一个错误——否则，如果名字自动填了默认值，某些错误可能会在发现之前就为时已晚。这意味着在给计数器累加之前必须先把它初始化为零，在扩展列表之前必须先把列表初始化为空，诸如此类。Python 中你不需要声明 (declare) 变量，但必须先把它们赋值，之后才能取它们的值。如今 Python 的其它错误消息会尽量提供更多帮助，比如 "Did you...?" (你是不是想...?) 式的提示——就像前面提到找 sys.ps1 这个提示符钩子时那样。使用 sys 这类模块之前必须先导入它 (不过很快你可能就不再需要这个提醒了，而且为了简洁，本书会截断一些错误消息)：

```
>>> sys.ps1 # 需要先 "import sys"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'sys' is not defined. Did you forget to import 'sys'?

```

所有这些你以后都会了解更多。这里的重点是：用这种方式犯错，你不会让 Python 或你的电脑崩溃。相反，你会得到一条有意义的错误消息，指出错误之处和产生错误的那行代码，然后你可以继续你的会话或脚本。事实上，一旦你熟悉了 Python，它的错误消息常常就能提供你需要的几乎全部调试支持 (更多调试选项请看侧边栏 "调试 Python 代码" (Debugging Python Code))。

深度理解

·核心概念——"先用后声明"原则 (Assignment before Use)：Python 的变量不需要声明 (declaration)，但必须先赋值才能取值。访问未定义的名字会抛 NameError。这一设计杜绝了"隐式初始化为 0/空"带来的隐藏 bug

·为什么这样设计：C 语言里使用未初始化变量是未定义行为（垃圾值）。Python 选择"一刀切报错"——宁可严格报错，也不允许静默地给你一个假默认值，因为"隐式默认值"会让 bug 潜伏到无法定位。这是显式优于隐式（explicit is better than implicit）哲学的又一次体现。

·Traceback 的结构：错误消息从上到下是——错误类型（如 NameError）、出错的代码位置（哪个文件哪一行哪个作用域）、以及具体描述。<stdin> 表示代码来自标准输入（即交互式键盘输入）。读到 Traceback 的顺序是"从下往上看主线索、从上往下看调用链"。

·NameError ... Did you forget to import'sys'?：Python 3.10+ 会给常见错误加"温馨提示"——这是解释器在尝试猜测你的意图（名字拼写错、忘了 import 等）。它是编译器级别的启发式功能，来自 CPython 的 "did you mean" 逻辑（也用于属性名拼写建议）。

·生活类比："变量必须先用后赋"就像"先开一盏灯再问它的亮度"——你不能问一个从未开过的灯有多亮，Python 干脆说"这灯我根本不认识"。

·初学者误区：①写 total = total + 1 却忘了初始化 total → NameError；②以为报错会"搞崩"程序——不会，当前这条语句失败，会话/脚本仍可继续；③把"未定义"误以为是"默认是 0"。

代码分析

```
>>> X          #
#
# Traceback (most recent call last):
#   File "<stdin>", line 1, in <module>
# NameError: name 'X' is not defined

>>> sys.ps1     # unimport
#
# Traceback (most recent call last):
#   File "<stdin>", line 1, in <module>
# NameError: name 'sys' is not defined. Did you forget to import '...
```

解释器如何处理：

1. 你在 REPL 中输入 X, Python 把它作为一个名字表达式编译。
2. 求值时, PVM 在当前作用域的命名空间 (这里是最外层/全局命名空间) 中查找名字 X。
3. 找不到 → 触发 NameError 异常。由于没有 try 捕获它, 默认异常处理器 (default exception handler) 接管, 打印 Traceback 到 stderr, 然后会话不中断地继续。
4. sys.ps1 同理, 但 Python 3.10+ 还附带了提示: 它会检查是不是存在"类似名字的模块/内置"来给你 Did you forget to import 'sys'? 的建议。
5. 教训: 用 sys 之前必须先 import sys —— import 会把模块对象的名字绑定到当前命名空间。

3.5.2 测试 (Testing)

英文原文

Besides serving as a tool for experimenting while you're learning the language, the interactive interpreter is also an ideal place to test code you've written in files. You can import your module files interactively and run tests on the tools they define by typing calls at the interactive prompt on the fly. For instance, the following tests a function in a precoded module that ships with Python in its standard library (detail: `os.getcwd` prints the name of the directory you're currently working in, here on a macOS host), but you can do the same once you start writing module files of your own:

```
>>> import os >>> os.getcwd() # Testing on the fly'
/Users/me/code'
```

More generally, the interactive prompt is a place to test program components, regardless of their source—you can import and test functions and classes in your Python files, type calls to linked-in C functions, exercise Java classes under Jython, and more. Partly because of its interactive nature, Python supports an experimental and exploratory programming style you'll find convenient. Although Python programmers also test with in-file code (and you'll learn ways to make this simple later in the book), for many, the interactive prompt is still their first line of testing defense.

中文翻译

除了作为你学习语言时做实验的工具，交互式解释器也是测试你写在文件里的代码的理想场所。你可以交互式地 `import`（导入）你的模块文件，然后随手在交互式提示符下输入对它们所定义工具的调用，运行测试。例如，下面测试了 Python 标准库中自带的一个预编码模块里的函数（细节：`os.getcwd` 打印你当前工作目录的名字，这里是 macOS 主机），但一旦你开始编写自己的模块文件，也可以做同样的事：

```
>>> import os >>> os.getcwd() # 随时测试'/Users/me/code'
```

更一般地说，交互式提示符是测试程序组件的地方，无论它们的来源是哪种——你可以导入并测试 Python 文件里的函数和类、输入对链接进来的 C 函数的调用、在 Jython 之下驱动 Java 类，等等。部分由于它的交互特性，Python 支持一种实验性、探索性的编程风格，你会发现它很方便。虽然 Python 程序员也会用文件内代码做测试（本书稍后会教你一些让它变简单的方法），但对许多人来说，交互式提示符仍然是他们测试的第一道防线（first line of testing defense）。

深度理解

- 核心概念：交互模式是“测试工作台”——`import` 把模块加载进来，然后你手动调用其函数、检查返回值。
- 为什么这个测试流程高效：Python 模块运行后，其顶层的绑定（函数、类、变量）会保留在模块对象中。你在 REPL 里 `import` 后，就获得了对模块 `dict`（命名空间）访问的入口，于是可以反复尝试各种参数组合。这正是“脚本语言快速迭代”的优势。

·底层细节：import os 会先搜索 os.py（标准库），把它编译成字节码、存入 pycache，然后执行其中全部顶层代码，最后把 os 名字绑定到模块对象。

·初学者误区：①os.getcwd() 必须带括号——它是函数调用，不带括号得到的是函数对象本身；②忘了 import os 直接用 os → NameError（3.5.1 刚演示过）；③以为交互式测试能代替正式测试——它能验证“函数能跑”，但自动化回归测试仍需文件内测试工具（如 unittest/pytest）。

代码分析

```
import os          # os
os.getcwd()        # os  getcwd
```

解释器如何处理：

1. import os 是语句。CPython 按模块搜索路径（sys.path：当前目录 → PYTHONPATH → 标准库目录 → site-packages）找到 os.py，首次导入时编译为字节码，把生成的 .pyc 缓存进 pycache/，然后执行模块顶层代码（os 是平台相关模块，内部还会再 import nt 或 posix 等具体实现模块）。

2. os.getcwd()：先对 os.getcwd 做属性解析（attribute lookup）——在 os 模块对象的命名空间里找到 getcwd 这个函数对象，然后以零个参数调用它。该函数在底层调用操作系统的 getcwd 系统调用，返回当前工作目录的字符串。

3. REPL 自动显示函数返回值'/Users/me/code'。测试方

法论：以后你写的每个模块，都可以这样 import 后逐个调用函数验证——这是 Python 程序员"第一道测试防线"

。

3.6 程序文件 (Program Files)

英文原文

Although the interactive prompt is great for experimenting and testing, it has one big disadvantage: programs you type there go away as soon as the Python interpreter executes them. Because the code you type interactively is never stored in a file, you can't run it again without retyping it from scratch. Cut-and-paste and command recall can help some here, but not much, especially when you start writing larger programs. To cut and paste code from an interactive session, you would have to edit out Python prompts, program outputs, and so on—which is too tedious to try. To save programs permanently, you need to write your code in files, which are usually known as modules. Modules are simply text files containing Python statements. Once they are coded, you can ask the Python interpreter to execute the statements in such a file any number of times, and in a variety of ways—by system command lines, by file icon clicks, by options in the IDLE user interface, and more. Regardless of how it is run, Python executes all the code in a module file from

top to bottom each time you run the file. Terminology in this domain can vary by role. For instance, module files are often referred to as programs in Python—that is, a program is considered to be a series of preordered statements stored in a file for repeated execution. Module files that are run directly are also sometimes called scripts—an informal term usually meaning a top-level program file. Some reserve the term module for a file imported from another file, and script for the main file of a program; we generally will here to (stay tuned for more on the meaning of top-level, imports, and main files later in this chapter). Whatever you call them, the next few sections explore ways to run code typed into files. In this section, you'll learn how to run files in the most basic and portable way: by listing their names in a Python command line entered at your computer's system prompt. Though this might seem primitive to some—and can often be avoided altogether by using alternatives discussed later—for many programmers, a system console window or command lines, together with a text editor window, constitutes as much of an integrated development environment as they will ever need and provides more direct control over programs.

中文翻译

虽然交互式提示符很适合实验和测试，但它有一个大缺点：你在那里输入的程序，一旦被 Python 解释器执行就消失了

。因为你交互式输入的代码从未存进文件，你想再运行它，就必须从头重新输入一遍。复制粘贴和命令召回（command recall）能帮上一点忙，但帮助不大，尤其是当你开始编写更大的程序时。要把代码从交互式会话中剪切粘贴出来，你必须编辑掉 Python 提示符、程序输出等等——这太繁琐了，不值得试。要永久保存程序，你需要把代码写进文件（files），它们通常被称为模块（modules）。模块就是包含 Python 语句的文本文件。一旦写好，你就可以让 Python 解释器以任意次数、多种方式执行文件中的语句——通过系统命令行、点击文件图标、IDLE 用户界面中的选项，等等。无论以何种方式运行，每次运行文件时，Python 都会从头到尾（from top to bottom）执行模块文件里的全部代码。这一领域的术语可能因角色而异。例如，模块文件在 Python 中常被称为程序（programs）——即程序被看作存储在文件里、供重复执行的一系列预编码语句。直接运行的模块文件有时也被称为脚本（scripts）——一个非正式术语，通常指顶层程序文件。有些人把术语 module 保留给“从另一个文件导入的文件”，把 script 留给“程序的主文件”；本书通常也这么用（稍后本章会继续讲顶层、导入和主文件的含义，敬请期待）。无论你叫它什么，接下来的几节会探索运行“输入到文件里的代码”的各种方式。在本节中，你将学到最基础、最可移植的运行方式：在计算机的系统提示符下输入一条 Python 命令行，在命令中列出它们的文件名。虽然对某些人来说这看起来原始——而且常常可以用后面讨论的替代方案完全避开——但对许多程序员而言，一个用于命令行的系统控制台窗口，加上一个文本编辑器窗口，就已经构成了他们永远需要的全部集成开发环境，并且提供了对程序更直接的控制。

深度理解

·核心概念——module 与 script 的精确区分：

· module (模块) = 任何以 .py 结尾的源文件，可以被其它文件 import。

· script (脚本) = 作为主文件/顶层文件直接运行的程序文件。

·同一个文件，被 import 时是"模块"，被直接运行时是"脚本"。角色由运行方式决定，不是文件本身的性质。

·"文件内没有自动回显"：前面强调过——REPL 会自动打印表达式结果，模块文件不会。文件里必须显式 print，否则输出为零。这是初学者第一个"脚本怎么没输出"的根源。

·"从头到尾执行"意味着什么：每次运行文件，解释器从上到下依次执行每条语句。这意味着：函数必须先定义后调用（运行时查找）；文件顶层的副作用（如 print）每次运行都会执行。

·为什么命令行+编辑器是老手的最爱：因为直接（explicit）——没有 GUI 的"糖衣"，你能精确控制运行目录、参数、环境变量。Lutz 坦言对很多人来说这就是"永远需要的 IDE"。

·生活类比：REPL 是黑板——粉笔字擦了就没了；模块文件是笔记本——记下来才能反复用、分发给别人（import）。

·初学者误区：①把 >>> 提示符连同代码一起存进 .py 文

件 → 语法错误；②以为 import 的文件"应该"像主脚本一样重复执行——不是，默认只执行一次（3.11.2 会讲 reload）。

3.7 第一个脚本 (A First Script)

英文原文

Let's get started. Open your favorite text editor, type or copy/paste the statements in Example 3-1 into a new text file named script1.py, and save it in your working code directory that you set up earlier (make it now if you skipped over that step). Any editor will work, including vi, Notepad, a smartphone app's editor, and the IDLE GUI coming up soon. You can also find this file in the book examples package, but typing code is an important exercise early on. Example 3-1. script1.py

```
python # A first Python script
import sys # Load a library module
print(sys.platform)
print(2 ** 100) # Raise 2 to a power
x = 'Hack!'
print(x * 8) # String repetition
```

This file is our first official Python script (not counting the two-liner in Chapter 2). You shouldn't worry too much about this file's contents just yet, but as a brief description, its code:- Imports a Python module (libraries of additional tools) to fetch the name of the platform- Runs three print function calls to display the script's results- Uses a variable named x, created when it's assigned, to hold on to a string object- Appli

es various object operations that we'll begin exploring in the next chapter. The `sys.platform` here is just a string that identifies the kind of computer you're working on; it lives in a Python module called `sys` (part of its standard library), which you must import to load (again, more on imports later). Also notice how this file uses explicit print calls; unlike the REPL, output in files is never automatic, so you must say `print` in files to see their output (and forgetting this is a regular first mistake, but at least you've been warned!). For color, this file adds some Python comments—the text after the `#` characters. These were mentioned earlier, but should be more formal now that they're showing up in scripts. Comments can show up on lines by themselves, or to the right of code on a line. The text after a `#` is simply ignored as a human-readable note and is not considered part of the statement's syntax. If you're copying this code, you can ignore the comments; they are just informative. This book uses a different formatting style to make comments more visually distinctive, but they'll be normal text in your code. Again, don't focus on the syntax of the code in this file for now; you'll learn about all of it later. The main point to notice is that you've typed this code into a file, rather than at the interactive prompt. In the process, you've coded a fully functional Python script. Notice, though, that the module file is named `script1.py`. As for all top-level files (i.e., files run directly), it could also be n

amed simply script1, but files of code you want to import in another file or REPL have to end with a .py suffix. Because you may want to import them in the future, it's a good idea to use .py suffixes for most Python files that you code. Also, some text editors and file explorers detect Python files by their .py suffix; if the suffix is not present, you may not get features like syntax colorization and automatic indentation in editors or tap-to-run in explorers.

中文翻译

让我们开始吧。打开你喜欢的文本编辑器，把示例 3-1 中的语句输入（或复制粘贴）到一个名为 script1.py 的新文本文件里，然后保存到你之前建立的代码工作目录中（如果跳过了那一步，现在就建立）。任何编辑器都可以，包括 vi、记事本（Notepad）、智能手机应用的编辑器，以及马上要讲到的 IDLE GUI。你也可以在本书示例包里找到这个文件，但早期亲自打字是一个重要练习。示例 3-1. script1.py

```
python # 第一个 Python 脚本
import sys # 加载一个库模块
print(sys.platform) print(2 ** 100) # 计算 2 的 100 次方
x = 'Hack!' print(x * 8) # 字符串重复这个文件是我们第一个正式的 Python 脚本（不算第 2 章那个两行的小例子）。你现在还不用太担心这个文件的内容，但简单描述一下，它的代码：
```

- 导入一个 Python 模块（额外工具的库），以获取平台的名字
- 运行三个 print 函数调用来显示脚本的结果
- 使用一个名为 x 的变量（在赋值时创建）来保存一个字符串对象
- 应用各种对象操作，下一章我们将开始探索这些操作

这里的 sys.platform 只是一个标识你正在使用的计

算机类型的字符串；它存在于一个名为 `sys` 的 Python 模块里（标准库的一部分），你必须先导入它才能加载（再说一次，关于导入后面还有更多内容）。还要注意这个文件是如何使用显式 `print` 调用的；与 REPL 不同，文件里的输出从来不是自动的，所以在文件里你必须写 `print` 才能看到输出（忘记这一点是常见的第一类错误，不过至少你已经被警告过了！）。为了增加色彩，这个文件添加了一些 Python 注释（comments）——即 `#` 字符之后的文本。这些在前面提过，但现在既然它们出现在脚本里，就应该正式认识一下。注释既可以独占一行，也可以出现在一行代码的右侧。`#` 之后的文本会被当作给人看的说明而直接忽略，不被视为语句语法的一部分。如果你在复制这段代码，可以忽略注释；它们只是信息性的。本书用一种不同的排版样式让注释更醒目，但它们在代码里就是普通文本。再说一次，现在别把注意力放在这个文件代码的语法上；你以后会全部学到。主要值得注意的点是：你是把这段代码输入到文件里，而不是在交互式提示符下。在这个过程中，你已经编写了一个功能完整的 Python 脚本。但请注意，这个模块文件名为 `script1.py`。和所有顶层（top-level）文件（即直接运行的文件）一样，它也可以只叫 `script1`；但你想在另一个文件或 REPL 里 `import` 的代码文件，必须以 `.py` 后缀结尾。因为你将来很可能想导入它们，所以对你编写的大多数 Python 文件，使用 `.py` 后缀是个好主意。另外，一些文本编辑器和文件浏览器会凭 `.py` 后缀识别 Python 文件；如果没有这个后缀，你可能就享受不到编辑器的语法高亮（syntax colorization）、自动缩进，或文件浏览器的点击即运行（tap-to-run）等功能。

深度理解

·核心概念：.py 后缀是 Python 世界的"身份证"——它决定了：①文件能否被 import 找到（import 只认 .py）；②编辑器是否启用 Python 语法高亮；③文件系统是否把文件关联给 Python 解释器。文件后缀在 Python 里不是摆设，是语言规范的一部分。

·为什么 import 必须认 .py：import 语句要按名字（如 sys）搜索文件，它会把名字和 .py 拼接去找 sys.py。没有后缀，搜索系统就没法工作。这是"命名即协议"（convention over configuration）的体现。

·脚本的三个功能块：本脚本其实展示了 Python 程序的三个基本构件——①import 加载库；②变量绑定（x='Hack!')；③函数调用与表达式（print(...)、2 100、x 8）。第 4 章起逐个展开。

·sys.platform 是什么：sys 模块在解释器启动时由 C 代码初始化，platform 是其中记录宿主平台的字符串（win 32/darwin/linux）。它是程序判断运行环境的惯用手法。

·生活类比：.py 后缀就像信封上的"邮政编码"——没有它，信件（import 搜索）就投递不到，编辑器也"看不出"这是 Python 代码。

·初学者误区：①文件名用 script.py 之外却存成 script.txt，导致 import 失败；②把文件命名为 sys.py 或 os.py——这会遮蔽（shadow）标准库同名模块，引发神秘错误！不要用标准库名字给你的文件命名。

代码分析

```
# Python
import sys                #
print(sys.platform)       # win32 / darwin / linux
print(2 ** 100)           # 2 100 Python
x = 'Hack!'               # x
print(x * 8)              # 8
```

解释器如何处理：

1. 逐行编译执行：解释器把整个文件编译为字节码（首次运行时存入 `pycache/script1.cpython-312.pyc`），然后由 PVM 从上到下逐条执行——先执行 `import sys`，再执行三条 `print` 与赋值。
2. `import sys`：执行 `import` 机制（搜索 → 编译 → 执行 → 绑定名字 `sys` 到当前模块的全局命名空间）。
3. `print(sys.platform)`：属性访问（attribute access）——从 `sys` 模块对象中取出 `platform` 字符串，作为 `print` 的参数输出。
4. `2 ** 100`：整数幂运算。Python 3 的 `int` 是任意精度的（基于 30 位小数的分段表示），所以 `2 ** 100` 能精确算出 31 位十进制数 1267650600228229401496703205376——无需担心 C 语言里的溢出。
5. `x = 'Hack!'`：赋值创建变量。`x` 指向内存中的一个 `str` 对象（对象在堆上分配，引用计数管理生命周期）。
6. `print(x * 8)`：`str.mul` 执行重复拼接，产生新字符串后传给 `print`。

7. 与 REPL 的关键区别：文件中第 4 行 `x = 'Hack!'` 不会有任何输出；第 3 行若没有 `print` 也什么都看不到——文件模式无自动回显。

3.8 用命令行运行文件 (Running Files with Command Lines)

英文原文

Once you've saved the preceding section's text file, you can ask Python to run it by listing its full filename as the first argument to a Python command—like the following typed at the system shell's \$ prompt on a Unix device (but don't type this at a Python REPL prompt, and read on to the next paragraph if this doesn't work right away for you):

```
$ python3 script1.py darwin
1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!
```

Just as for starting the REPL we studied earlier, the command name may vary (e.g., use `py` instead of `python3` on Windows, usually), and you can type such a command in whatever your system provides for command-line entry—Command Prompt on Windows, Terminal on macOS, or the Termux app on Android, among others. You might also run this code file with a dedicated run button in a GUI, app, or browser-based UI, but we'll postpone such options until later in this chapter (see also Appendix A for all things platform specific). Be sure to run yo

ur Python command in the same working directory where you've saved your script file (cd there first if needed), and run it at the system prompt, not Python's >>> prompt. Also like starting the REPL, you may have to replace the command's first word with a full directory path if Python isn't on your PATH setting, but most installs automatically ensure that it is (see Appendix A for more on Python installs and PATH). If all works as planned, this shell command (or similar) makes Python run the code in this file line by line, and you will see the output of the script's three print statements—the name of the underlying platform as known to Python, 2 raised to the power 100, and the result of the same string repetition expression we saw earlier (there's more on the meaning of the last two of these in Part II). If all didn't work as planned, you'll get an error message—make sure you've entered the code in your file exactly as shown, and try again. We'll talk about debugging options in the sidebar "Debugging Python Code", but at this point in the book your best bet is probably rote imitation. And if all else fails, you might also try running under the IDLE GUI discussed ahead—a tool that sugarcoats some launching details, though sometimes at the expense of the more explicit control you have when using command lines. If copying code grows too tedious or error-prone, you can also fetch this book's examples on the web, though again, typing code initially will help you learn to av

oid syntax errors. See the Preface for info on obtaining the examples.

中文翻译

保存好上一节的文件后，你可以通过把完整的文件名作为 Python 命令的第一个参数列出来，让 Python 运行它——就像下面这样，在类 Unix 设备的系统 shell \$ 提示符下输入（但不要把它输入到 Python REPL 提示符里；如果你没有立刻成功，请读一下下一段）：`$ python3 script1.py darwin 1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!` 和前面学习启动 REPL 时一样，命令名可能不同（例如在 Windows 上通常用 `py` 而不是 `python3`），你可以在系统提供的任何命令行入口里输入这样的命令——Windows 的命令提示符（Command Prompt）、macOS 的终端（Terminal）、Android 的 Termux 应用，等等。你也可以在 GUI、应用或浏览器 UI 中用专用的运行按钮运行这个代码文件，但我们把这些选项推迟到本章后面（平台相关的所有细节也请看附录 A）。请务必在与脚本文件保存位置相同的工作目录里运行你的 Python 命令（必要时先 `cd` 过去），并在系统提示符下运行，而不是在 Python 的 `>>>` 提示符下。和启动 REPL 一样，如果 Python 不在你的 PATH 设置里，你可能得把命令的第一个词替换成完整目录路径；但大多数安装会自动确保它在（更多关于 Python 安装和 PATH 的内容见附录 A）。如果一切按计划进行，这条 shell 命令（或类似命令）会让 Python 一行一行地运行文件里的代码，你将看到脚本三条 `print` 语句的输出——Python 所知的底层平台名、2 的 100 次方、以及我们之前见过的那条字符串重复

表达式的结果（后两者的含义在第二部分还有更多内容）。如果一切没有按计划进行，你会得到一条错误消息——请确保文件中的代码与你所见的完全一致，然后重试。我们会在侧边栏“调试 Python 代码”（Debugging Python Code）里讨论调试选项，但在这个阶段，你最好的策略大概是机械模仿（rote imitation）。如果一切都不奏效，你也可以试试在后面的 IDLE GUI 下运行——那是一个把某些启动细节“裹上糖衣”的工具，不过有时要以牺牲使用命令行时的更显式控制为代价。如果复制代码变得越来越繁琐或容易出错，你也可以从网上获取本书的示例；不过再说一次，初期亲手打字能帮你学会避免语法错误。获取示例的信息见前言（Preface）。

深度理解

- 核心概念——“解释器 + 文件名”模式：python3 script1.py 的意思是“用 Python 3 解释器执行 script1.py 这个文件”。解释器读完整个文件后逐行执行。这是脚本语言最经典的运行方式。
- 第一个参数（first argument）：python3 是程序名，script1.py 是它的第一个命令行参数。解释器看到参数就以“脚本模式”启动——不再进入 REPL，而是打开文件、按文件内容执行。
- 工作目录的重要性：命令必须在“脚本所在目录”运行（或给完整路径）。因为解释器要找文件、脚本内部要读写文件，都相对当前工作目录进行。cd 是 shell 命令，不是 Python 命令。

- 为什么不能在 `>>>` 里输入：`python3` 是 shell 的程序名，不是 Python 语句；在 `>>>` 里输入它会导致 `SyntaxError`。两个世界（shell 与 Python）的命令不能混用。
- 2 100 的输出在第二部分才讲解：作者在这里埋点——整数、字符串是后面 Part II 的主角，现在只需“看得懂输出”。
- 生活类比：`python3 script1.py` 就像“请大厨（`python3`）按菜谱（`script1.py`）做菜”——菜谱要放在灶台（工作目录）上，大厨才能找到。
- 初学者误区：①在 `>>>` 里输入 `python3 script1.py`；②双击 `.py` 文件一闪而过看不到输出（3.10.1 会讲原因）；③文件名拼写错误（`script1.py` vs `script1.py`）导致 `can't open file` 报错。

代码分析

```
# shell Python
python3 script1.py # Windows py script1.py
```

解释器如何处理：

1. shell 解析命令：程序名 `python3`、参数 `script1.py`。
2. 操作系统按 PATH 找到 `python3` 可执行文件，启动新进程，并把 `script1.py` 作为 `argv[1]` 传给进程。
3. CPython 启动后检查 `argv`：没有参数 → REPL 模式；有参数 → 把它当作脚本文件路径，进入脚本模式（`script mode`）。
4. 脚本模式下，解释器读取整个文件、编译为字节码（若

有 pycache 则直接加载缓存以省去编译)、然后从上到下执行全部语句。

5. 三行输出对应脚本里的三个 print——这就是"运行文件的命令行"的全部秘密: 换一个入口, 执行逻辑完全相同。

3.9 命令行用法变体 (Command-Line Usage Variations)

英文原文

When you type a command to run a Python code file, the command you type is run by a system shell program (e.g., Bash on Unix). Because of this, all of the shell's syntax is available for more custom runs. For instance, you can route the printed output of a Python script to a file to save it for later use or inspection, by using special shell syntax:

\$ python3 script1.py > saveit.txt In this case, the three output lines shown in the prior run are stored in the file saveit.txt instead of being printed. This is generally known as stream redirection; it works for both output (>) and input (<) text and is available on Windows and Unix-like systems. This is useful for testing, as you can write programs that watch for changes in other programs' outputs. It also has little to do with Python, though (Python simply supports it), so we will skip further details on shell redirection syntax here. Re

direction is for command lines only, though, because it's a function of the system shell. On Windows, you can also type just the name of your script and omit the name of Python itself. Because Windows uses filename associations to find a program with which to run a file, the file's name is enough to run a .py file. The following command, for example, will automatically be run by Python on Windows (technically, by Python's py Windows launcher described in Appendix A):

```
$ script1.py
```

This works just as though you had clicked on the file's icon in File Explorer (a launch mode covered later). One fine point here: Command Prompt runs programs this way in its own window, but PowerShell may not; use `py` to view unredirected output in the latter if needed (or use the `icon-click input()` trick also coming up). Finally, remember to give the full path to your script file if it lives in a different directory than the one in which you are working. For example, the `py` Python command in the following, run in PowerShell on Windows, assumes Python is in your system path but runs a file located elsewhere:

```
PS C:\Users\me\code> cd D:\temp\savecode # Go to a different folder
PS D:\temp\savecode> py C:\Users\me\code\script1.py # Run a script elsewhere
win32 1 267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!
```

If your `PATH` doesn't include Python's directory, and

neither Python nor your script file is in the directory you're working in, use full paths for both—like the following on macOS, which for good measure throws in output stream redirection to a file located outside the current working directory (where you are when this command is run):

```
$ /usr/local/bin/python3 /Users/me/code/script1.py  
> /Users/me/data/saveit.py
```

This is a lot to type, but a lot pathological and atypical. To keep your commands simpler than this, make sure Python is on your PATH, and cd to script or data folders first. Again, most installs set up PATH automatically, so you need only focus on script and data folders when running commands.

中文翻译

当你输入命令运行一个 Python 代码文件时，你输入的命令是由系统的 shell 程序（例如 Unix 上的 Bash）执行的。正因如此，shell 的全部语法都可用来做更定制的运行。例如，你可以用特殊的 shell 语法，把 Python 脚本打印的输出路由到一个文件，保存起来供以后使用或检查：`$ python3 script1.py > saveit.txt` 这种情况下，前一次运行中显示的三行输出会被存进 `saveit.txt` 文件，而不是打印出来。这通常被称为流重定向（stream redirection）；它对输出（`>`）和输入（`<`）文本都有效，在 Windows 和类 Unix 系统上都可用。这对测试很有用，因为你可以编写程序来监视其它程序输出的变化。不过它与 Python 本身关系不大（Python 只是支持它），所以我们这里跳过 shell 重定向语

法的更多细节。不过重定向只适用于命令行，因为它是系统 shell 的功能。在 Windows 上，你还可以只输入脚本的名字，而省略 Python 本身的名字。因为 Windows 使用文件关联（filename associations）来寻找运行某个文件的程序，所以对于 .py 文件，光有文件名就足以运行它。例如，下面的命令在 Windows 上会自动由 Python 运行（严格说，是由附录 A 中描述的 Python py Windows 启动器运行）：`$ script1.py` 这就像你在文件资源管理器中点击了文件图标一样（后面会讲这种启动模式）。这里有一个细节：命令提示符（Command Prompt）会在自己的窗口里用这种方式运行程序，但 PowerShell 可能不会；在后者中，如果需要查看未重定向的输出，就用 `py`（或者用后面要讲的图标点击 `input()` 技巧）。最后，如果你的脚本文件所在的目录与你当前工作的目录不同，请记得给出脚本文件的完整路径。例如，下面在 Windows 的 PowerShell 中运行的 `py` 命令，假定 Python 在你的系统路径中，但运行的是位于别处的文件：`PS C:\Users\me\code> cd D:\temp\savecode # 转到另一个文件夹`
`PS D:\temp\savecode> py C:\Users\me\code\script1.py # 运行别处的脚本`
win32 1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack!
如果你的 PATH 里没有 Python 的目录，而且 Python 和你的脚本文件都不在你工作的目录里，那就给两者都用完整路径——就像下面在 macOS 上那样，它顺手还把输出流重定向到了当前工作目录（你运行命令时所在的位置）之外的一个文件：`$ /usr/local/bin/python3 /Users/me/code/script1.py > /Users/me/data/saveit.py` 这样要输入很多东西，但也算病态和非典型的用法。要让命令比这更简单，请确保 Python 在 PATH 里

，并先 cd 到脚本或数据文件夹。再说一次，大多数安装会自动配置好 PATH，所以运行命令时你只需要关心脚本和数据文件夹。

深度理解

·核心概念：命令行由 shell 解释执行，因此你可以白赚 shell 的能力——重定向 (>/<)、管道 (|) 等，都是"零成本"的额外功能。Python 只是被当作一个普通的命令行程序。

·重定向的底层：> 是 shell 层的文件描述符操作——把进程的标准输出 (stdout, fd 1) 指向文件，而不是终端。这是操作系统功能，与 Python 无关。Python 的 print 写 stdout，所以被重定向走。

·Windows 文件关联 (filename association)：Windows 注册表把 .py 扩展名关联到 Python。输 script1.py 时，cmd 调用关联程序 (py launcher) 运行它。这和双击图标是同一机制。

·PowerShell 的怪癖：PowerShell 对"直接输入文件名"的处理与 cmd 不同（可能把输出当作对象管道化而不显示），所以作者建议用 py 保持确定性。

·完整路径的两种使用场景：①Python 不在 PATH——给解释器全路径；②脚本不在当前目录——给脚本全路径。两者都缺才需要都写全。

·生活类比：shell 重定向就像"开会时有人做会议纪要"——原本打在屏幕上的字（输出）被转写进文档（文件），发言人（Python）根本不知道换了记录方式。

·初学者误区：①把 `>` 当 Python 语法用（在 REPL 里输入 `print(...) > file`）——那是 shell 的事；②忘记 Windows 的 `script1.py` 直跑依赖文件关联，在 PowerShell 里行为不同；③路径含空格不引号导致找不到文件。

3.10 运行文件的其它方式 (Other Ways to Run Files)

英文原文

If you're not a fan of command lines, you can generally avoid them by launching Python scripts with file icons, development GUIs, and other schemes that vary per platform and role. This book generally recommends command-line usage for learners, both because it's simple, and because it's a common, general, portable, and powerful way to run code. But it's not required. While the Python world is too rich in options for exhaustive coverage here, let's take a quick tour of the most prominent command-line alternatives to close out this chapter.

中文翻译

如果你不喜欢命令行，通常可以避开它——用点击文件图标、开发 GUI 以及其它随平台和角色而异的方案来启动 Python 脚本。本书总体上向学习者推荐命令行用法，因为它既简单，又是常见、通用、可移植且强大的运行代码方式。但这不是必须的。虽然 Python 世界的选项丰富到无法在此

详尽覆盖，但为了给本章收尾，我们快速浏览一下最突出的命令行替代方案。

深度理解

·核心概念：命令行的替代品很多（图标点击、GUI、IDE、App、浏览器），它们本质都是“用不同 UI 包装同一个解释器”。底层运行的还是 python 解释器，只是入口换了。

·为什么作者坚持推荐命令行：简单（一条命令）、通用（所有平台都有 shell）、可移植（不会绑定某家 IDE）、强大（可组合 shell 语法）。GUI 方案则各有学习曲线和平台绑定。

·学习建议：不要跳过命令行直接投奔 IDE——命令行暴露的“显式控制”（工作目录、参数、环境变量）是理解 Python 运行机制的最快路径。

3.10.1 点击/轻点文件图标 (Clicking and Tapping File Icons)

英文原文

On most PC platforms, Python program files can be run by simply clicking or tapping their filename or icon in the local file explorer. For example, this works automatically on Windows in File Explorer, thanks to file name associations set up during Python's install. Clicks also run code files on macOS in Finder, if made to Open With the Python Launcher included in the mac

OS install; drags to the Python Launcher app when available work the same as clicks. The file-clicks story is more involved on Linux, where files likely need executable permission and a `#!` first line to name Python. On smartphones, tapping a filename in a file explorer on Android (or Files on iOS) may open the file in an associated Python app, but this may work only for some explorer/app combos, and may not give access to all of a program's files in some contexts. For more insight, consult your platform and app docs, or experiment on your device. Also see Appendix A for more info. As noted there, Windows and Linux clicks don't keep the window open for viewing output and error messages after programs end: if a script just prints and exits, it, well, just prints and exits—the console window appears, and text is printed there, but the console window closes and disappears on program exit. Per Appendix A, coding a closing call to Python's `input()` forces a pause before exit so you can see output, but this doesn't help for error messages. Use other run techniques when this matters.

中文翻译

在大多数 PC 平台上，只需在本地文件资源管理器中点击或轻点文件名或图标，就能运行 Python 程序文件。例如，在 Windows 的文件资源管理器（File Explorer）里，这能自动生效——多亏了 Python 安装时建立的文件关联（filename associations）。在 macOS 的 Finder 里，如果把文

件"用打开方式 (Open With) "选择为 macOS 安装自带的 Python Launcher, 点击也能运行代码文件; 在可用时把文件拖到 Python Launcher 应用上, 效果与点击相同。Linux 上的文件点击故事更复杂: 文件很可能需要可执行权限 (executable permission) 和以 `#!` 开头的第一行来指明 Python。在智能手机上, 在 Android 的文件资源管理器 (或 iOS 的"文件 Files"应用) 中轻点文件名, 可能会在关联的 Python 应用里打开该文件; 但这可能只对某些资源管理器/应用组合有效, 而且在某些情况下可能无法访问程序的所有文件。想了解更多, 请查阅你的平台和应用文档, 或在你的设备上亲自实验。更多信息也请看附录 A。正如那里所述, 在 Windows 和 Linux 上点击运行, 程序结束后不会让窗口保持打开以查看输出和错误消息: 如果一个脚本只是打印然后退出, 那么它——呃——就是打印然后退出; 控制台窗口出现、文本被打印到那里, 但程序退出时控制台窗口就关闭消失了。按附录 A 的说法, 在代码末尾调用一次 Python 的 `input()` 会强制在退出前暂停, 让你看到输出; 但这帮不上错误消息的忙 (错误发生在到达 `input()` 之前)。当这一点很重要时, 请使用其它运行技术。

深度理解

·核心概念: 点击图标 = 隐式调用"关联程序" (Windows) 或"Python Launcher" (macOS)。这套机制方便, 但隐藏了运行细节, 还带来窗口一闪而过的老问题。

·为什么窗口会闪退: 双击运行脚本时, 系统为脚本开一个控制台窗口; 脚本执行完、进程退出, 窗口随之关闭。如果脚本只是 `print` 完就退出, 你根本来不及看输出。这不是 Python 的 bug, 是控制台窗口生命周期决定的。

- 两个经典解决方案：
- 脚本末尾加 `input()`——让程序等待用户输入，窗口停住直到按回车。但只对正常运行有效：若中途报错，异常已使进程退出，到不了 `input()`。
- 更可靠：改用命令行或 IDE 运行——输出和错误留在终端/IDE 面板里。
- Linux 的 `#! + chmod`：在 Unix 系，直接运行文件需要两个条件——文件有可执行位 (`chmod +x`)，且首行 `#!/usr/bin/env python3` 告诉内核用哪个解释器。这是 Unix 的"可执行文本文件"协议。
- 生活类比：双击图标就像按了"自动洗车"按钮——方便，但你想中途看泡沫怎么冲的（看输出），洗完了车（窗口）已经开走了。
- 初学者误区：①双击 `.py` 报错闪退就以为"程序坏了"——先在命令行跑看错误信息；②把 `#!` 当作 Python 语法（它是给操作系统看的注释）；③以为所有平台双击行为一致。

3.10.2 IDLE 图形用户界面 (The IDLE Graphical User Interface)

英文原文

So far, we've seen how to run Python code with interactive sessions, system command lines, and icon clicks. If you're looking for something a bit more visual, IDLE provides a GUI for Python programming, and it's

a standard and free part of the Python system. IDLE is usually referred to as an integrated development environment (IDE), because it binds together multiple development tasks into a single view. In short, IDLE lets you edit, run, browse, and debug Python programs, all from the same GUI. Because it's coded in Python with the tkinter GUI toolkit, it runs portably on all Python PC platforms—Windows, macOS, and Linux. For many, IDLE represents an easy-to-use alternative to typing command lines, a less problem-prone alternative to clicking on icons, and a great way for newcomers to get started editing and running code. You'll sacrifice some control in the bargain, but this typically becomes important only later in your Python career. IDLE install and startup are covered in Appendix A, so we won't repeat the full details here. In brief, it's standard with the python.org Python installers for Windows and macOS and can be had separately in Linux repositories. Once installed, it can be launched with the usual suspects: Start on Windows, Launchpad or Finder on macOS, a command line on Linux, and file right-clicks where supported. Appendix A also has screenshots that capture IDLE in action on each platform, but Figure 3-1 captures its dark theme on Windows for both variety and a quicker look; as it demos, after running this chapter's script1.py from Example 3-1 in the edit window on top, its output appears in the interactive Shell window. > Figure 3-1. IDLE with its dark the

me on Windows > Tip: because IDLE is just a Python script on the module search path in the standard library, you can also generally run it on any platform and from any directory by typing the following Python command in a system console window (use `py` instead of `python3` on Windows, as usual). Python's `-m` flag simply locates a module using the normal import search, but runs it as a top-level script (Part V covers both this search, and the `"."` package syntax required here):

```
> > > $ python3 -m idlelib.idle # Find and run idle.py in a package folder
```

> Once IDLE is started, its usage is straightforward and documented in its in-program Help. In brief, the Shell window provides the usual interactive REPL with command recall, and each edit window allows you to view, modify, and run a file of code. The Shell's `File → New File` and `File → Open...` start and open code files. Edit windows' `Run → Run Module` runs code in the window where it's selected, and `Run → Run...` Customized supports passing command-line arguments to scripts (which is out of scope here and not a full system shell, but useful for scripts that expect these). IDLE has many more features, but as a sampler: it also does tab completions, pop-up balloon help for functions, and object attribute lists as you type code, and includes an object browser and GUI debugger. To use IDLE's debugger, enable it in the `Debug` menu, set breakpoints by right-clicks in edit windows, and run. For simpler debugging needs, right-

click on the text of any error message in the Shell window to jump to the line of code where the error occurred. For more IDLE tips, see this book's Appendix A, IDLE's own Help menu, and the notes for your platform in "Python Setup and Usage" in Python's standard manuals. Like most GUIs, the best way to learn IDLE may be to test-drive it for yourself. At the end of the day, its usability may be essential for some beginners, but it comes with an extra learning curve, is not as flexible as command lines, and adds sugarcoating that might be a negative when your programming needs outgrow its scope. As always, vet for yourself on a PC near you.

中文翻译

到目前为止，我们已经看到用交互式会话、系统命令行和点击图标运行 Python 代码的方式。如果你想要更"可视化"一点的东西，IDLE 为 Python 编程提供了一个 GUI，它是 Python 系统标准且免费的一部分。IDLE 通常被称为集成开发环境 (integrated development environment, IDE)，因为它把多种开发任务整合进单一视图。简言之，IDLE 让你在同一个 GUI 里编辑、运行、浏览和调试 Python 程序。由于它是用 Python 配合 tkinter GUI 工具包编写的，它能在所有 Python PC 平台上可移植地运行——Windows、macOS 和 Linux。对许多人来说，IDLE 是输入命令行之外的易用替代品，是点击图标之外更不易出错的替代品，也是新手开始编辑和运行代码的好方式。代价是你牺牲了一些控制力，不过这通常要到你 Python 生涯后期才变得重要。

IDLE 的安装和启动在附录 A 中覆盖，这里不重复全部细节。简言之，它是 python.org 的 Windows 和 macOS 安装器标配，在 Linux 仓库里也可以单独获取。安装后，可以用常规手段启动：Windows 的开始 (Start) 菜单、macOS 的 Launchpad 或 Finder、Linux 的命令行，以及支持处的文件右键菜单。附录 A 还有 IDLE 在各平台运行时的截图，但图 3-1 捕捉了它在 Windows 上的深色主题，既多样又直观；如图所示，在上方编辑窗口中运行本章示例 3-1 的 script1.py 之后，其输出出现在交互式 Shell 窗口中。>

图 3-1. Windows 上使用深色主题 IDLE 的提示：因为 IDLE 本身只是标准库模块搜索路径上的一个 Python 脚本，你通常也可以在任意平台、任意目录下，通过在系统控制台窗口输入下面的 Python 命令来运行它（Windows 上照例用 py 代替 python3）。Python 的 -m 标志只是用常规的导入搜索来定位一个模块，但把它作为顶层脚本运行（第五部分既会讲这种搜索，也会讲这里所需的 "." 包语法）：

```
> > > $ python3 -m idlelib.idle # 在包文件夹中找到并运行 idle.py
```

> IDLE 启动后，它的用法很直白，并在程序内 Help 里有文档。简言之，Shell 窗口提供带命令召回的常规交互式 REPL，每个编辑窗口让你查看、修改和运行一个代码文件。Shell 的文件 → 新建文件 (File → New File) 和文件 → 打开... (File → Open...) 用来新建和打开代码文件。编辑窗口的运行 → 运行模块 (Run → Run Module) 运行当前选中窗口里的代码，运行 → 运行...自定义 (Run → Run... Customized) 支持向脚本传递命令行参数（这超出本书范围，也不是完整的系统 shell，但对期待这类参数的脚本有用）。IDLE 还有更多功能，作为抽样：它支持制表符补全 (tab completions)、函数的弹出式气球

帮助 (balloon help)、边输入代码边列出对象属性，还包含对象浏览器 (object browser) 和 GUI 调试器。要使用 IDLE 的调试器，在 Debug 菜单中启用它，在编辑窗口里右键设置断点 (breakpoints)，然后运行。对更简单的调试需求，在 Shell 窗口里右键点击任何错误消息的文本，即可跳到出错的那行代码。更多 IDLE 技巧见本书附录 A、IDLE 自己的 Help 菜单，以及 Python 标准手册中 "Python Setup and Usage" 里针对你平台的说明。和大多数 GUI 一样，学习 IDLE 的最好方式可能是亲自试驾。到头来，它的易用性对某些初学者可能是必需的，但它自带一条额外的学习曲线，不如命令行灵活，还添加了一层"糖衣" (sugarc coating) —— 当你的编程需求超出它的范围时，这层糖衣可能变成负资产。一如既往，请在你附近的 PC 上亲自验证。

深度理解

- 核心概念：IDLE 的定位是"零门槛 IDE"——编辑、运行、调试三合一，随 Python 免费分发。它由 tkinter 编写，本身就是 Python 程序 (所谓"吃自己的狗粮")。
- IDLE 的架构：两个核心窗口——Shell 窗口 (REPL，即交互式解释器界面) 和编辑窗口 (多文件文本编辑器)。Run → Run Module 相当于把编辑窗口的文件作为顶层脚本交给解释器执行，输出回到 Shell 窗口。
- `python3 -m idlelib.idle` 的深层含义：`-m` 让 Python 用 import 搜索机制找模块并当作脚本运行。`idlelib` 是包 (package)，`idle` 是包里的模块，`.` 是包层级语法。这预示了 Part V 的模块系统——运行模块和导入模块共用

同一套文件搜索逻辑。

·为什么要看深色主题截图：图 3-1 同时示范了"编辑窗口 + Shell 窗口"的双窗口布局——理解这个布局就理解了 IDLE 的一切操作。

·IDLE 的取舍：对新手，GUI 隐藏了路径、参数等细节，降低入门门槛；对老手，这些"被隐藏的细节"正是需要显式控制的东西。作者态度中立："vet for yourself"（自己验证）。

·生活类比：IDLE 就像"一体式豆浆机"——泡豆、研磨、煮浆一步到位，但不灵活（你不能单独调节研磨细度）；命令行像"分开的豆浆机+锅"，麻烦但每一步都可控。

·初学者误区：①以为 IDLE 是"Python 的一部分必须用"——它只是可选工具；②在 Shell 窗口输 `python3 xxx.py`——Shell 里不需要也不接受这个，直接用 `Run → Run Module`；③不知道 `Run → Run Module` 是运行"当前编辑窗口"的文件，运行错文件还一脸茫然。

代码分析

```
# Python
python3 -m idlelib.idle # IDLE
```

解释器如何处理：

1. `-m` 告诉解释器："接下来的参数是模块名，不要当文件名，去 `sys.path` 里搜索它。"
2. 搜索找到 `idlelib` 包（标准库目录下的一个文件夹），再找到包内的 `idle` 模块（`idlelib/idle.py`）。

3. 解释器像运行顶层脚本一样执行该模块，并设置 `name == 'main'`，于是 IDLE 的 GUI 主循环启动。
 4. 意义：这证明了“模块”与“脚本”的同一性——同一份文件，被 `import` 时是模块，被 `-m` 运行时是脚本。这是本书 Part V 的核心思想，在这里首次露出端倪。
-

3.10.3 其它 Python IDE (Other IDEs for Python)

英文原文

Because IDLE is free, portable, and a standard part of Python, it's a nice first choice to become familiar with if you want to use an IDE at all. There are, however, a handful of alternative IDEs for Python developers, some of which are substantially more powerful and robust than IDLE.

Among these, PyCharm, PyDev, Wing, VSCode, Spyder, and PyScripter all come with the usual edit-and-run GUIs, but some also add additional and advanced tools such as code refactoring and source-control integration. These IDEs also come with major learning curves and are not recommended for most Python beginners.

Still, you may wish to file this away for later in your Python career, when you've mastered the language and move on to industrial-strength development.

中文翻译

因为 IDLE 免费、可移植、又是 Python 的标准组成部分，如果你确实想用 IDE，它是个很好的初次上手选择。不过，面向 Python 开发者的替代 IDE 也有一批，其中一些比 IDLE 强大得多、健壮得多。

其中，PyCharm、PyDev、Wing、VSCode、Spyder 和 PyScripter 都自带常规的"编辑+运行"GUI，但有些还额外增加了高级工具，比如代码重构（code refactoring）和源码控制集成（source-control integration）。这些 IDE 同时也有着陡峭的学习曲线（learning curves），对大多数 Python 初学者并不推荐。

不过，你或许可以把这个信息"存档"备用——等到你的 Python 生涯后期，当你已经掌握这门语言、迈向工业化级（industrial-strength）开发时再搬出来用。

深度理解

·核心概念：IDLE 的定位是"够用的入门 IDE"；它之上还有真正的"重型武器"——PyCharm、VSCode、Wing、Spyder 等。作者的态度是：现在记住名字即可，别急着装。

·为什么"不推荐给初学者"：这些 IDE 功能强，代价是学习曲线陡峭。它们默认隐藏大量细节（虚拟环境管理、调试器、重构工具、源码控制），初学者会被层层 UI 干扰，难以建立对"解释器到底怎么跑我的代码"的直觉。作者建议先用 REPL 和命令行把语言本身学会。

· "industrial-strength development" (工业级开发) 意味着什么：在大项目里，编译式重命名、跨文件重构、git 集成、单测夹具等工具能力直接决定团队效率——那时重型 IDE 的价值才兑现。

· 实际建议：新手阶段用"轻、稳、透明"的工具（命令行 + IDLE，或后期 VSCode）；当你的项目跨多个文件、需要调试器断点、需要重构时，再上 PyCharm/VSCode 并投入时间学习它。

· 生活类比：学开车先用教练车的驾校教材思路——IDLE 像科目一教材，PyCharm 像 F1 赛车的仪表盘——车是好车，但新手上去只会被仪表盘吓到。

· 初学者误区：①听说"PyCharm 最强"就去装，结果被设置面板劝退，Python 语法一个字没学；②以为必须用 IDE 才能写 Python——完全不是，命令行 + 任意文本编辑器即可。

3.10.4 智能手机应用 (Smartphone Apps)

英文原文

If you're running this book's examples on a smartphone or tablet with Android or iOS, you'll be using an application. Some, like Termux on Android, come with a traditional command line and support all the REPL and file commands we've seen. In others, you'll launch files of code with taps and other tools in the app's user interface (e.g., Actions, Run buttons), instead of

traditional command lines. This isn't much different from launching in IDLE or other IDEs, but the details vary by app, so see Appendix A for mobile-specific tips, and your app's docs for more info.

中文翻译

如果你是在 Android 或 iOS 的手机、平板上运行本书示例，你会用到某个应用。有些应用，比如 Android 上的 Termux，自带传统命令行，我们前面见过的全部 REPL 和文件命令都能用。但在其它应用里，你是用应用用户界面上的控件（比如点击按钮）来启动代码文件，而不是传统命令行。这在精神上和在 IDLE 或其它 IDE 里运行代码没有多大不同，只是细节因应用而异——所以移动平台的小贴士请看附录 A，更多信息请看你的应用文档。

深度理解

·核心概念：智能手机上的 Python 运行方式分为两大阵营——“真命令行”（Termux 这类，把 PC 体验搬到手机）与“GUI 化 App”（Pydroid、Pythonista 这类，用按钮/界面隐藏一切细节）。前者把完整技能搬到手机，后者牺牲细节换取易用。

·底层差异：Termux 里你其实在运行一个完整的 Linux 环境（python、mkdir、shell 都齐），写什么代码和 PC 上一模一样；GUI App 则把“文件系统、目录、命令行参数”这些概念都藏起来，这在初学阶段反而可能让人困惑。

·为什么存在 App 这种形态：Python 的“免费随处可跑”哲学延伸到了移动端——手机对很多人来说是唯一拥有的”

电脑”。《Learning Python》仍努力做到“在任何设备上都能跟学”。

·实际问题：移动端最大的坑在于——文件放在哪、App 能不能访问、import 能不能定位同目录模块。建议把代码放在 App 官方指定的目录（如 Termux 的 ~/ 或 Pydroid 的主目录），并优先用自带终端。

·生活类比：Termux 像“手机上的真电脑”（小屏+键盘），GUI App 像“家用打印机控制面板”——能干，但选项固定。

·初学者误区：①以为装了 App 就有完整文件系统权限——很多 App 出于安全原因只让你访问固定目录；②在 App 里找不到 >>> 提示符就以为不能交互——多数 App 提供“终端页签”。

3.10.5 浏览器里的 WebAssembly (WebAssembly for Browsers)

英文原文

Although still an emerging technology, it's also possible to run Python code in web browsers. This is enabled today by WebAssembly (a.k.a. Wasm), which defines a portable bytecode format that is run by web browsers, much as the Python PVM runs its own bytecode (see Chapter 2). By compiling the Python interpreter's source code to this format with tools like the Emscripten LLVM-based compiler, web browsers are able to

o run Python, and hence, your Python programs. While other Python-in-the-browser initiatives of the past have largely fizzled, WebAssembly is standardized by the World Wide Web Consortium (W3C) and already supported by all major desktop and mobile web browsers.

Compiling Python itself to Wasm is not trivial, and much too much to ask of Python beginners, but the Pyodide system has already done much of the work for you. It's a port of CPython to the WebAssembly/Emscripten platform, compiled and ready to run, with JavaScript integration for access to the browser DOM and web APIs. To use Pyodide in a web page, you'll load it from a server and initialize it in a browser, with an HTML document that uses supplied JavaScript code and API tools. The end result can run many Python programs in web browsers, with no local installs required.

The chief downsides of this model seem to be speed and utility. Downloading a Python interpreter to run a Python script isn't fast, and the speed of Python scripts can be slow relative to natives. Moreover, Pyodide comes with a fixed set of Python tools, and Python scripts run by browsers live in a sandbox with limited access to tools and resources on the host device. Persistent storage, for example, may support POSIX file calls and paths, but is virtual and ultimately supported by browsers. Hence, while this option may avoid some Python installs, it opens the possibility of Python

on the web, but with limitations for general software development, and futures options are impossible to predict. Watch the web for more on this evolving story—including the alternative MicroPython for Wasm, which is smaller than CPython but implements a constrained subset of the language, and the py2wasm Python-to-Wasm compiler, announced just as this book was being written.

中文翻译

虽然仍是一项新兴技术，但现在也可以在网页浏览器里运行 Python 代码。今天这由 WebAssembly（又名 Wasm）实现——它定义了一种可移植的字节码（bytecode）格式，由网页浏览器运行，正如第 2 章里 Python 虚拟机（PVM）运行自己的字节码一样。通过用 Emscripten 这类基于 LLVM 的编译器把 Python 解释器的源码编译成这种格式，网页浏览器就能运行 Python，也就运行了你的 Python 程序。过去“把 Python 跑在浏览器里”的其它倡议大多不了了之，但 WebAssembly 由万维网联盟（World Wide Web Consortium, W3C）标准化，并且已得到所有主流桌面和移动浏览器的支持。

把 Python 本身编译成 Wasm 并不简单（让 Python 初学者去做更是强人所难），但 Pyodide 系统已经替你完成了大部分工作：它是 CPython 到 WebAssembly/Emscripten 平台的一个移植，编译好即可运行，并带 JavaScript 集成，可访问浏览器的文档对象模型（DOM, Document Object Model）和 Web API。要在网页里使用 Pyodide，你要从一个服务器加载它，并在浏览器里用一个使用现成 J

JavaScript 代码和 API 工具的 HTML 文档来初始化它。最终结果是：无需任何本地安装，就能在浏览器里运行许多 Python 程序。

这个模式的主要缺点似乎是速度（speed）和实用性（utility）。下载一个编译好的 CPython 解释器到浏览器里运行一个 Python 脚本并不快，而在此环境中 Python 脚本的速度也可能不如人意。此外，Pyodide 自带固定的一套 Python 工具，而浏览器运行的 Python 脚本又活在一个沙盒（sandbox）里，对宿主设备上的工具和资源访问受限。例如，持久化存储（persistent storage）可能支持 POSIX 文件调用和路径，但它是虚拟的，最终只支持浏览器所支持的选项。所以，虽然这个选项能省去一些 Python 安装、为 Web 上的 Python 打开新的可能，但它对通用软件开发并没有其它方式那么有用，其未来也无法预测。请在网络上关注这个仍在演进的故事——包括另一种针对 Wasm 的 MicroPython（比 CPython 小，但实现的是第 2 章所述的受限 Python 子集），以及一个在本书写作时刚刚宣布的 py2wasm Python 到 Wasm 编译器。

深度理解

·核心概念：浏览器里也能跑 Python，靠的是 WebAssembly（Wasm）——一种浏览器原生支持的“可移植字节码”。这把 Python 的“字节码→解释器”模型又套了一层：Python 字节码 → 解释器 → Wasm 字节码 → 浏览器。

·底层如何实现：Wasm 是浏览器这门“虚拟机”的执行格式。用编译器（如 Emscripten）把 CPython 解释器（本身是 C 写的）编译成 Wasm，浏览器就能当“解释器的宿主”

运行，从而解释 Python 源码。Pyodide 就是"预编译好的 CPython for Wasm + JS 胶水"。这就是设备无需安装 Python 的原因——解释器跟着网页下载。

·为什么是"沙盒"：浏览器有严格的安全模型 (browser security model) ——沙盒限制对资源 (文件、网络等) 的访问。所以 Pyodide 里"打开文件"其实是浏览器虚拟出来的。浏览器的安全模型决定了 Python 在这里只能是"受限"：能计算、能画图、能交互，但不能随心所欲地操作主机系统。

·实际用处：这类方案适合"无需安装环境就能演示 Python"——"把 Python 代码放进网页给用户玩"，不适合做真正的后台服务或本地大型项目。对真正的软件工程来说，命令行/IDE 仍然不可替代。

·生活类比：Wasm 里的 Python 像"来你家帮忙的钟点工"——能干不少活 (算数、画图表)，但你不能把"装修布线" (控制系统资源) 这种粗活交给它。

·初学者误区：①以为 Wasm 的 Python 就是"普通 Python"——它跑在浏览器沙盒里，能用的模块有限；②把它当作"免安装好方案"——它的启动速度其实很慢；③读到"浏览器能跑 Python"就想到"以后不用装环境了"——错了，这种模式的适用范围很窄。

3.10.6 面向科学的 Jupyter 笔记本 (Jupyter Notebooks for Science)

英文原文

This is another launch option we met earlier, in conjunction with REPLs. Jupyter is a set of tools that allow Python code to be run in web browsers, with a focus on supporting scientific-programming tasks. Its primary and classic tools require a server to be separately installed and launched to run the code you enter in a web page, but it also comes in a form that runs code locally in browsers using the WebAssembly and Pyodide systems described in the preceding section. In both forms, Jupyter pages follow a flexible notebook paradigm: interactive coding using the IPython REPL we also met earlier, code cells that run with a button click, visualization using Python numeric tools, and more .

While Jupyter is a useful and popular tool in many STEM (science, technology, engineering, and math) roles, it's not targeted at general-purpose software development and isn't as broadly applicable as traditional tools like the command lines stressed in this book. See other resources for usage details if Jupyter notebooks may be a part of your Python coding future.

中文翻译

这个我们之前在谈 REPL 时已经遇见的选项。Jupyter 是一组让 Python 代码能在网页浏览器中运行的工具，重点支持科学编程（scientific-programming）任务。它的经典工

需要一个单独安装并启动的服务器 (server) 来运行你在网页里输入的代码；不过它还有一种形式，是使用上一节描述的 WebAssembly 和 Pyodide 系统在浏览器本地直接运行代码。无论哪种形式，Jupyter 页面都遵循灵活的笔记本 (notebook) 范式——用我们前面也见过的 IPython REPL 交互式编码、一点按钮就运行的代码单元格 (code cells)、用 Python 数值工具做可视化，等等。

虽然 Jupyter 在许多科学、技术、工程和数学 (STEM) 岗位上是好用又流行的工具，但它并不针对通用软件开发，也不像本书强调的命令行这类传统工具那样广泛适用。如果 Jupyter 笔记本可能成为你未来 Python 编码生涯的一部分，请参阅其它资源了解详细用法。

深度理解

·核心概念：Jupyter = 浏览器 + IPython 内核 + 笔记本文档。它本质上是"把交互过程保存成可复现、可分享的文档"。一篇 notebook = 叙述文本 + 可执行的代码单元格 + 程序输出。

·底层架构：浏览器页面只是"前端"，真正执行代码的是后台运行的一个 IPython 内核（一个长期存活的 Python 进程）。所以单元格之间共享变量（单元格内的状态保留）——这是 notebook 的"力量"也是新手最容易迷失的地方（单元格执行顺序依赖）。

·为什么它适合科学计算：科学工作流是"试算 → 看图 → 改参数 → 再算"，notebook 的"单元格 + 可视化"恰恰贴合这个模式。而工程软件开发是"源码文件 → 版本控制 → 构建 → 部署"，命令行 + 编辑器才是正统工作流。两种工

作品格不同。

·实际场景：数据分析（统计、画图）、机器学习实验记录、教学演示、报表生成。它不是通用“程序运行环境”，而是“叙述性计算文档”。

·生活类比：notebook 像“实验室的工作日志”——每一步实验（单元格）都记录并可可视化，翻到哪页都能接着做；而脚本文件像“交给工厂的图纸”——要一版一版地走完整流程。

·初学者误区：①以为 Jupyter 是 Python 的“标准运行方式”——它只是科学计算场景的一个工具；②不管单元格执行顺序埋头跑——输出的数字很容易“神秘消失”；③拿它跑后端服务代码——那是 IDE/命令行的职责。

3.10.7 把代码编译成机器码：AOT 编译器 (Ahead-of-Time Compilers for Speed)

英文原文

Also per Chapter 2, a number of ahead-of-time (AOT) compiler systems—including Nuitka and Shed Skin—compile Python programs all the way to machine code, like C and C++ compilers compile C and C++ programs. Once you install such a system, you'll run its compiler against your file of Python code, and it will produce an executable program. You run that executable like any other program. AOT compilers can speed up your program, but at the cost of additional steps t

that slow the development process and negate some of the advantages of using Python. They're probably best saved for later in your career, when speed truly matters.

中文翻译

按照第 2 章的说法，一些提前编译（ahead-of-time, AOT）的编译器系统——包括 Nuitka 和 Shed Skin——可以把 Python 程序一直编译成机器码（machine code），就像 C 和 C++ 编译器把 C/C++ 程序编译成机器码一样。你一旦安装这类系统，先要把你的 Python 代码文件交给它的编译器，然后像运行其它可执行程序那样运行编译结果。AOT 编译器可以提升程序速度，但代价是增加了额外的开发步骤、明显拖慢编程流程，并抵消了使用 Python 的一些优势。对 Python 新手来说，最好在通过常规 REPL、IDE、命令行这类更易上手的途径学完 Python，之后再探索这些系统。

深度理解

·核心概念：之前讲的都是“解释型运行”（每次运行都解释字节码）；AOT 走的是一次性编译成机器码的 C/C++ 路线，编译产物是“可执行文件”，以后直接运行。用“编译时间”换“运行速度”。

·底层原理：Nuitka 把 Python 源码翻译成 C 代码再调用编译器编译成原生可执行文件；Shed Skin 则在推断类型后把代码编译成 C 再到机器码。但有一个现实的困难：Python 的动态类型 + 反射特性使其“编译成高效机器码”远

比 C/C++ 艰难——这也是为什么即便 AOT 后的 Python 仍难达到 C 的性能。

·为什么牺牲开发速度也要用：当你的 Python 脚本是“每天处理海量数据的批任务”时，运行速度直接决定成本和可行性，巨大的收益压倒编译成本；但若程序需要频繁迭代，编译时间反而会拖垮开发效率。

·与解释执行的 Python 的关系：这不是“更好”，而是“另一类工具”。开发期用解释器（快速获得反馈），需要极致性能的发布期才考虑 AOT——两者在 Python 生态中长期并存。

·初学者误区：①以为“把 Python 编译成独立可执行文件”就等于“更快”——编译优化的是性能，语义仍然不变；②指望靠 AOT 轻易拿到 C 的速度；③以为第 2 章讲的“解释器”和 AOT“编译”互斥——其实 Python 生态里编译器和解释器并存，组合使用才是常态。

·生活类比：AOT 像“预制菜”——备料（编译）一次，开饭（运行）很快；解释型像“现炒现卖”——每顿都从切菜（解释）开始，灵活但慢。两者做的是同一道菜（同样的 Python 语义），只是“备料时机”不同。

3.11 在代码中运行代码 (Running Code in Code)

英文原文

This chapter has talked about "importing modules" a few times without really explaining what this term means. We'll study modules and larger program architecture in depth in Part V, but because imports are also a way to launch programs, this section will sketch in some basic module and import concepts now.

The short story is that Python code can also be launched by Python code—that is, you can run a file or a string of code from within another file or a REPL, using tools like `import`, `exec`, and standard-library launchers. Let's take a first look at all three, before we close out this chapter.

中文翻译

本章已经好几次提到"导入模块 (importing modules)"，却没有真正解释这个术语的意思。我们会在第五部分深入钻研模块和更庞大的程序架构，但因为 `import` 也是一种启动程序的方式，本节现在先把一些基本的模块和导入概念讲个大概。

简而言之，Python 代码也可以由 Python 代码来启动——也就是说，你可以在另一个文件或 REPL 内部，用 `import`、`exec` 和标准库启动器等工具，运行一个文件或一段代码字符串。在收尾本章之前，我们先对这三者做个初探。

深度理解

·核心概念：至此"启动"的概念被推进到终极形态——用代码启动代码。之前是人（敲命令、点图标）启动程序；这一节开始，程序自己也能启动其它代码：import 启动文件、exec 启动代码字符串、os.system/subprocess 启动命令行进程。

·为什么要讲这些：import 是 Python 程序架构的基石（Part V 才展开），exec 是动态执行的万能钥匙，os.system 是程序内"再造一个命令行"。三者都是在第 2 章"解释器模型"之上的合法启动入口。

·生活类比：之前你像"老板"一样手点外卖（命令行/图标）；这一节教你写一个"自动订餐程序"——程序调用程序，一环扣一环。

·初学者误区：以为"运行程序"只有"人敲命令"一种方式；实际上操作系统、其它程序、甚至网页都在不停地"启动"Python 代码。

3.11.1 导入模块 (Importing Modules)

英文原文

In simple terms, every file of Python source code whose name ends in a .py extension is a module. No special code or syntax is required to make a file a module: any such file will do. Other files can access the items a module defines by importing it—which essentially I

loads another file and gives you access to that file's contents. The contents of a module are made available to the outside world through its attributes—a term defined informally in the next section.

This module paradigm turns out to be the core idea behind program architecture in Python. Larger programs usually take the form of multiple module files, which import tools from other module files. One of the modules is designated as the main or top-level file, also often called the script—the file launched to start the entire program, which runs line by line as usual. Below this level, it's all modules importing modules.

We'll delve into such architectural issues in more detail later in this book. This chapter is mostly interested in the fact that import operations run the code in a file being loaded as a final step. Because of this, importing a file is yet another way to launch it. For instance, if you start an interactive session (from a system command line or otherwise), you can run the script1.py file we wrote earlier in Example 3-1 with a simple import statement—which is really Python code running other Python code:

```
$ python3 >>> import script1 darwin 126765060022  
8229401496703205376 Hack!Hack!Hack!Hack!Hack!H  
ack!Hack!Hack!
```

> NOTE. Where to run imports: Be sure to run this Python command in the directory (i.e., folder) containin

g script1.py. Per this chapter's earlier coverage of code folders, this is easiest if you save module files and run imports in a per-chapter folder. Later in this book, you'll learn that imports in a REPL search for a module in the current directory, plus those listed on environment variable PYTHONPATH or specified otherwise. The current directory part of this will suffice for most imports you're likely to try until then—as long as all your code files reside there.

中文翻译

简单地说，每个文件名以 .py 扩展名结尾的 Python 源代码文件都是一个模块（module）。不需要任何特殊的代码或语法来让一个文件成为模块：任何这样的文件都可以。其它文件可以通过导入（importing）来访问模块定义的内容——导入本质上就是加载另一个文件，并让你访问该文件的内容。模块的内容通过它的属性（attributes）向外界开放——这个术语下一节会有非正式的定义。

这种模块范式最终被证明是 Python 程序架构（program architecture）背后的核心思想。较大的程序通常表现为多个模块文件，彼此互相导入工具。其中一个模块被指定为主文件或顶层（top-level）文件，也常被称为脚本（script）——即被启动以开启整个程序的文件，它照常一行一行地运行。在它之下，全部是“模块导入模块”。

我们稍后会在本书中更详细地钻研这些架构问题。本章主要关心的事实是：导入操作会把被加载文件中的代码作为最后一步运行。正因如此，导入一个文件是启动它的另一种方式。例如，如果你启动一个交互式会话（从系统命令行或其它

方式)，你可以用一条简单的 `import` 语句运行我们之前在示例 3-1 里写的 `script1.py` 文件——这其实就是"Python 代码在运行其它 Python 代码"：

```
$ python3 >>> import script1 darwin 126765060022  
8229401496703205376 Hack!Hack!Hack!Hack!Hack!H  
ack!Hack!Hack!
```

> 注意。在哪儿运行导入：请务必在包含 `script1.py` 的目录（即文件夹）里运行这条 Python 命令。按照本章前面关于代码文件夹的讲解，最省事的方式是：把你正在学习的章节的模块文件和导入操作都放在一个按章节划分的文件夹里。本书后面你会学到：REPL 中的导入会按当前目录（`current directory`）、环境变量 `PYTHONPATH` 中列出的目录、或以其它方式指定的目录来搜索模块。在那之前，只要你把所有代码文件都放在当前目录下，仅靠"当前目录"就足以应对你很可能尝试的大部分导入。

深度理解

- 核心概念：`.py` 文件 = 模块，`import` = 执行文件内容并暴露其属性。这里第一次正式定义"模块"：不需要任何特殊声明，凡是 `.py` 文件都是模块。
- "导入运行文件"的双重身份：`import script1` 干了两件事——①搜索并执行 `script1.py` 的全部顶层代码（所以三个 `print` 都输出了）；②把模块对象以名字 `script1` 绑定到当前命名空间。这是"运行程序"的又一种官方姿势。
- 为什么 `import` 要在当前目录运行：`import` 的搜索路径（`sys.path`）第一项通常是当前工作目录。你在别的目录

import script1 会报 ModuleNotFoundError——这就是本章前面强调"代码文件夹"的回报。

·程序架构蓝图：Python 大型程序的形态 = 一个顶层脚本 + 一堆被它导入的模块。这是 Part V 的主线，这里只是"剧透"。

·生活类比：import 像"借邻居家的工具"——打开人家的工具箱（执行文件）、把工具摆出来（属性暴露），你就能借来用；而"直接运行脚本"像"把邻居请到你家现场表演一遍"——同样的事做一次，但不会有"工具借出"关系。

·初学者误区：①在错误目录 import 报错就以为代码坏了；②以为 import 只是"加载定义"而不会执行代码——不，导入时会执行文件的顶层语句（顶层 print 会打印！），这是初学者最意外的行为。

代码分析

```
# REPL Python
import script1 # = script1.py
```

解释器如何处理：

1. import script1 时，CPython 先按 sys.path 搜索 script1.py（当前目录排第一，所以能命中）。
2. 首次导入：把源码编译成字节码，缓存进 pycache/script1.cpython-312.pyc，然后执行整个文件的顶层代码——三个 print 依次输出，这就是你看到的 darwin、2100、字符串重复三行。
3. 执行完毕后，把模块对象绑定到当前命名空间的名字 script1。

4. 关键：这里没有出现任何“错误”，因为导入成功地把文件“运行”了一遍。程序员的直觉是“import = 读文件”，但 Python 的真相是“import = 执行文件 + 绑定模块对象”。

3.11.2 重新加载模块 (Reloading Modules)

英文原文

Imports work to run a file, but only once per session (really, process—a program run) by default. After the first import, later imports do nothing, even if you change and save the module's source file again in another window:

```
>>> import script1 >>> import script1
```

This is by design; imports are too expensive an operation to repeat more than once per file in a given program run. As you'll learn in Chapter 22, imports must find files, compile them to bytecode, and run the code line by line, and importers usually care only that the module's lines have defined its exports.

If you really want to force Python to run the file again in the same session without stopping and restarting the REPL, you need to instead call the reload function available in the `importlib` standard-library module (and previously in the now-defunct `imp` module, and a built-in function before that: that's three incarnations, for anyone counting!):


```
>>> from importlib import reload >>> reload(script
1) darwin 1267650600228229401496703205376 Hack
!Hack!Hack!Hack!Hack!Hack!Hack!Hack! <module'scri
pt1' from'/Users/me/Code/script1.py'>
```

The `from` statement here simply copies a name out of a module (more on this in the next section). The `reload` function itself loads and runs the current version of your file's source code, picking up the file's changes if you've modified and saved it in another window. This allows you to edit and use new code on the fly in the current Python interactive session.

The `reload` function expects the name of an already-loaded module object, so you have to have successfully imported a module before you can reload it (and if the import failed with an error, you can't yet reload, and must import again).

Notice that `reload` also expects parentheses around the module object name, whereas `import` does not—`reload` is a function that is called with an argument, and `import` is a statement. That's also why you get back an extra output line when reloading—the odd last line is just the display representation of the `reload` call's return value, a Python module object. You'll learn more about using functions in general in Chapter 16; for now, when you hear "function," remember that parentheses are required to run a call, even if there's nothing to send.

中文翻译

导入可以用来运行文件，但默认情况下每个会话（session）只做一次（准确地说，是每个进程 process——一次程序运行）。第一次导入之后，再导入就什么都不做了——即使你在另一个窗口修改并保存了模块的源文件：

```
>>> import script1 >>> import script1
```

这是有意设计：在给定的一次程序运行中，对每个文件重复导入是过于昂贵的操作。正如你在第 22 章会学到的，导入必须查找文件、把它们编译成字节码、逐行运行代码——而导入者通常只关心模块的代码行是否已经定义了它导出的内容。

如果你真的想在同一会话里强制 Python 再次运行该文件，而又不停止并重启 REPL，你需要改而调用 `importlib` 标准库模块中的 `reload` 函数（它之前存在于现已废弃的 `imp` 模块中，再之前则是个内建函数——整整三个化身，数一数！）：

```
>>> from importlib import reload >>> reload(script1)
darwin 1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!
<module'script1' from '/Users/me/Code/script1.py'>
```

这里的 `from` 语句只是把一个名字从模块中复制出来（下一节有更多内容）。`reload` 函数本身会加载并运行你文件的源代码的当前版本——如果你在另一个窗口修改并保存过文件，它会拾取这些改动。这让你能在当前 Python 交互式会话中即时编辑并使用新代码。

`reload` 函数期望的是已经加载的模块对象的名称，所以你必须先成功导入一个模块，之后才能 `reload` 它（如果导入因错误而失败，你还不能 `reload`，必须重新 `import`）。

注意 `reload` 还要求模块对象名字两边带括号，而 `import` 不用——`reload` 是一个需要用参数调用（call）的函数，而 `import` 是一条语句。这也解释了为什么 `reload` 时会多一行输出——那最后一行奇怪的输出，只是 `reload` 调用返回值的显示表示（display representation），它是一个 Python 模块对象。你会在第 16 章更全面地学习函数的使用；现在，听到“函数”时请记住：即使没什么可传，调用也必须带括号。

深度理解

·核心概念：`import` 默认每个进程只执行一次（这是“导入缓存”机制）；想在同一进程里强制重跑，用 `importlib.reload()`。

·为什么 `import` 只跑一次：因为 `import` 是“昂贵操作”——查找文件 → 编译字节码 → 执行。而且导入者的语义是“让我能用到这个模块的导出”，不是“每 `import` 一次就重跑一遍”。重复导入会浪费 CPU，还可能在重跑时重置模块内部状态。

·`reload` 的底层行为：`reload` 会重新执行模块的顶层代码，并就地更新已存在的模块对象（`sys.modules` 里的那个），而不是新建模块对象。注意：`reload` 只更新 `import` 绑定的名字，不会更新 `from ... import name` 已经复制出去的名字——这是著名“坑”。

- import 是语句、reload 是函数：语句和函数的调用语法不同——import a（无括号）vs reload(a)（带括号）。reload 的最后一行输出就是函数返回值的 REPL 自动回显（模块对象的 repr）。
- 历史彩蛋：reload 的"三个化身"——内置函数 → imp 模块 → importlib 模块，反映了 Python 标准库重构史（imp 在 3.12 中被移除）。
- 生活类比：import 像"安装好的家用电器"——装一次就一直在那儿，按开关（reload）才重新工作；再"安装"一遍什么也不会发生。
- 初学者误区：①改了文件后继续 import，以为会自动生效——不会！必须 reload 或重启进程；②用 from x import f 后又 reload——拿到的还是旧函数；③给 reload 不存在的模块名——必须先 import 成功。

代码分析

```
import script1          #
import script1          #
from importlib import reload  # importlib reload
reload(script1)         # script1
```

解释器如何处理：

1. 第一次 import script1：sys.modules 里没有 script1 → 走完整导入流程（查找→编译→执行→绑定）。
2. 第二次 import script1：sys.modules 里已经有 script1 → 直接把缓存对象绑定，不执行文件。
3. from importlib import reload：导入 importlib 模块

- ， 把其 reload 属性复制为当前作用域的名字 reload。
4. reload(script1): 函数调用。reload 从 sys.modules 取出 script1 模块对象，重新编译并执行它的源码，用新命名空间覆盖更新模块对象内部的名字；返回该模块对象，REPL 于是自动回显 <module'script1' from'...'>。
5. 设计思想：函数调用必须带括号 (reload(x))，语句不用 (import x) ——这种语法差异是有意为之，帮助区分 "执行动作" 与 "绑定关系"。
-

3.11.3 模块属性初探 (Module Attributes: A First Look)

英文原文

Imports and reloads provide a natural program launch option because import operations execute files as a last step. In the broader scheme of things, though, modules serve the role of libraries of tools, as you'll learn in Part V. The basic idea is straightforward, though: a module is mostly just a package of variable names, known as a namespace, and the names within that package are called attributes—variable names that are attached to a specific object (like a module). In more concrete terms, all the names assigned at the top level of a module's file become its attributes, and a module's importers gain access to all of them. These names are usually assigned to tools exported by the module—functions, classes, variables, and so on—that are

intended to be used in other files and other programs. Externally, a module file's names can be fetched with two Python statements, `import` and `from`, and may be reset with the `reload` call.

To illustrate, use a text editor to create a one-line Python module file called `myfile.py` in your working directory, with the contents in Example 3-2.

Example 3-2. `myfile.py`

```
python title = 'Learning Python, 6th Edition'
```

This may be one of the world's simplest Python modules (it contains a single assignment statement), but it's enough to illustrate the point. When this file is imported, its code is run to generate the module's attribute. That is, the assignment statement creates a variable and module attribute named `title`.

You can access this module's `title` attribute in other components in two different ways. First, you can load the module as a whole with an `import` statement, and then qualify the module name with the attribute name to fetch it (note that we're letting the interpreter print automatically here):

```
$ python3 # Start Python REPL >>> import myfile #  
Run file, load module as a whole >>> myfile.title # U  
se its attribute names: '.' to qualify 'Learning Python, 6  
th Edition'
```

In general, the dot expression syntax `object.attribute`

lets you fetch any attribute attached to any object and is one of the most common operations in Python code. Here, we've used it to access the string variable `title` inside the module `myfile`—in other words, `myfile.title`.

Alternatively, you can fetch (really, copy) names out of a module with `from` statements:

```
$ python3 # Restart Python REPL >>> from myfile i
import title # Run file, copy its names >>> title # Use
name directly: no need to qualify 'Learning Python, 6t
h Edition'
```

As you'll see in more detail later, `from` is just like an `import`, with an extra assignment to names in the importing code. Technically, `from` copies a module's attributes, such that they become simple variables in the recipient. So, you can refer to the imported string `this time as title` (a variable) instead of `myfile.title` (an attribute).

Naturally, modules usually define more than one name to be used both in and outside their files. Example 3-3, for instance, defines `three`.

Example 3-3. `threenames.py`

```
python a = 'PC' # Define three attributes b = 'Phone' #
Exported to other files c = 'Tablet' print(a, b, c) # Also
used as variables in this file This code file, threename
s.py, assigns three variables, and so generates three a
```

tributes for the outside world. It also uses its own three variables in a print statement, as you see when this is run as a top-level file from a system prompt:

```
$ python3 threenames.py PC Phone Tablet
```

All of this file's code also runs as usual the first time it is imported elsewhere, by either an import or from:

```
$ python3 >>> import threenames # Grab the whole module: it runs here PC Phone Tablet
```

```
>>> threenames.b, threenames.c # Access its attributes: qualify to use ('Phone','Tablet')
```

```
>>> from threenames import b, c # Copy multiple names out: use directly >>> b, c ('Phone','Tablet')
```

The results here are printed in parentheses because they are really tuples—a kind of object created by the comma in the inputs (and covered in the next part of this book)—that you can safely accept on faith for now.

From a grander perspective, modules form the highest layer of Python program architecture: as self-contained namespaces, they naturally support code organization and reuse, and they automatically minimize name collisions in your code. We'll deal with their loftier goals later in this book.

For this chapter, it's enough to know that imports and reloads are another way to run your code files, though probably a secondary option: due to complicate

d quirks that we'll skip here (e.g., reload updates imports using import but not from), tools like command lines and IDEs are generally better bets for running Python code.

中文翻译

导入和重载提供了一种天然的“程序启动”选项，因为导入操作把执行文件作为最后一步。不过从更广阔的视角看，模块承担的是工具库（libraries）的角色，正如你在第五部分会学到的。基本思想其实很直白：模块本质上是一个变量名的包裹（package），称为命名空间（namespace），包裹里的名字称为属性（attributes）——即附着在某个特定对象（比如模块）上的变量名。更具体地说，模块文件顶层赋值的所有名字都会成为它的属性，模块的导入者可以访问到所有这些属性。这些名字通常被赋值为模块导出的工具——函数、类、变量等等——供其它文件和其它程序使用。从外部看，模块文件的名称可以用 `import` 和 `from` 两条 Python 语句取用，并可以通过 `reload` 调用重置。

为了说明这一点，请用文本编辑器在你的工作目录里创建一个单行的 Python 模块文件 `myfile.py`，内容见示例 3-2。

示例 3-2. `myfile.py`

```
python title = 'Learning Python, 6th Edition'
```

这可能是世界上最简单的 Python 模块之一（它只包含一条赋值语句），但足以说明问题。当这个文件被导入时，它的代码会被运行，从而生成模块的属性。也就是说，这条赋值语句创建了一个名为 `title` 的变量，同时也成为模块属性。

你可以在其它组件中用两种不同的方式访问这个模块的 `title` 属性。第一种，用 `import` 语句把模块作为整体加载，然后用点号限定（qualify）模块名来取属性（注意这里我们让解释器自动打印）：

```
$ python3 # 启动 Python REPL >>> import myfile # 运行文件，整体加载模块>>> myfile.title # 使用它的属性名：用 '.' 来限定 'Learning Python, 6th Edition'
```

一般来说，点号表达式语法 `object.attribute`（对象.属性）让你能取出任何对象上附着的任意属性，它是 Python 代码中最常见的操作之一。这里我们用它在模块 `myfile` 内部访问字符串变量 `title`——也就是 `myfile.title`。

另一种方式，你可以用 `from` 语句从模块中取出（准确说是复制）名字：

```
$ python3 # 重启 Python REPL >>> from myfile import title # 运行文件，复制它的名字>>> title # 直接使用名字：无需限定 'Learning Python, 6th Edition'
```

正如你稍后会更详细看到的，`from` 就像 `import`，额外多做了一次对导入代码里名字的赋值。技术上，`from` 复制模块的属性，使它们变成接收方（recipient）里的普通变量。所以这次你可以直接用 `title`（一个变量）来指代导入的字符串，而不是 `myfile.title`（一个属性）。

自然，模块通常会定义不止一个名字，供文件内外共同使用。比如示例 3-3 就定义了三个。

示例 3-3. `threenames.py`

```
python a = 'PC' # 定义三个属性 b = 'Phone' # 导出到其它文件 c = 'Tablet' print(a, b, c) # 本文件内部也当变量用
```

这个代码文件 `threenames.py` 赋值了三个变量，因此为外界生成了三个属性。它还在一条 `print` 语句里用到了自己的三个变量——当你把它作为顶层文件从系统提示符运行时就能看到：

```
$ python3 threenames.py PC Phone Tablet
```

这个文件的所有代码，第一次在别处被导入时（无论是 `import` 还是 `from`）也会照常运行：

```
$ python3 >>> import threenames # 抓取整个模块：
这里它运行了PC Phone Tablet >>> threenames.b, thre
enames.c # 访问它的属性：用点号限定('Phone','Tablet')
>>> from threenames import b, c # 复制多个名字出来
：直接使用>>> b, c ('Phone','Tablet')
```

这里的结果被打印在括号里，是因为它们其实是元组 (tuples) ——一种由输入里的逗号创建的对象（本书下一部分会讲到）——现在你可以放心地先接受这个事实。

从更高的视角看，模块构成 Python 程序架构的最高层：作为自包含的命名空间，它们天然支持代码组织和复用，并自动最小化你代码中的名字冲突 (name collisions)。它们的更高目标，我们会在本书后面处理。

对本章来说，知道“导入和重载是运行代码文件的另一种方式”就够了——尽管这大概是个次要选项 (secondary option)：由于这里略过的一些复杂怪癖（例如 `reload` 会更新使用 `import` 的导入者，但不会更新使用 `from` 的导入者），命令行和 IDE 这类工具通常更适合运行 Python 代码。

深度理解

·核心概念：模块 = 命名空间（namespace）= 顶层名字的集合；属性 = 模块里的变量名。这是 Python 程序架构的第一课：.py 文件顶层赋的每一个名字，都自动成为该模块的属性。

·两种取用方式的本质差异：

·import myfile → 用 myfile.title 点号限定访问。模块仍是一个独立命名空间，名字不泄漏。

·from myfile import title → 复制一份名字进当前作用域，title 成为本地变量。省事，但切断了与模块的同步（reload 也不更新它）。

·为什么 from 是"复制"而不是"引用"：from 本质是"先 import，再把属性赋给本地名字"。之后你改 title 不影响 myfile.title；reload 模块也不会更新已复制的 title。这是后续 Part V 的重要知识点。

·顶层代码执行的后果：threenames.py 里的 print(a, b, c) 在导入时也会执行——所以 import threenames 输出了 PC Phone Tablet。模块顶层代码"每次被导入只执行一次"（本进程内）。

·程序架构的最高层：大型程序 = 顶层脚本 + 相互导入的模块；模块的命名空间隔离自动消除了名字冲突——两个模块都定义 a 互不干扰。这是 Python 天然的设计优势。

·生活类比：模块像"工具房的货架"——每个货架（模块）有自己的格子（命名空间）；import 是"站在货架前取用工具"（要报货架名）；from 是"把工具搬到自己家"（直

接用，但搬走了就与货架无关了）。

·初学者误区：①以为 `from ... import` 会"保持同步"——不会，它只是复制；②在模块里写顶层代码（如 `print`）时没意识到 `import` 时会执行；③两个文件互相 `import` 造成循环导入——Part V 会讲。

代码分析

```
# myfile.py
title = 'Learning Python, 6th Edition' # →

# threenames.py
a = 'PC' #
b = 'Phone' #
c = 'Tablet'
print(a, b, c) #

# REPL
import myfile # myfile.py myfile
myfile.title # → 'Learning Python, 6th E...
from myfile import title # title
```

解释器如何处理：

1. `import myfile`：搜索→编译→执行 `myfile.py` 顶层代码（执行 `title = ...`）→ 创建一个模块对象，其命名空间里存着 `title` → 把 `myfile` 绑定到当前命名空间。
2. `myfile.title`：点号表达式——先解析 `myfile` 找到模块对象，再在其命名空间（dict）里查 `title`。属性访问是 Python 中最常用的字节码操作之一（`LOADATTR`）。
3. `from myfile import title`：等价于"`import myfile`，然后 `title = myfile.title`"——把模块对象的 `title` 属性复

制到当前命名空间成为普通变量。

4. 设计思想：用命名空间隔离代码 = "显式优于隐式"的工程化体现——谁导入谁负责声明，冲突自消。

3.11.4 exec 内建函数 (The exec Built-In)

英文原文

Python also provides a way to launch files with code that does not rely on the module concepts of the preceding section. The `exec` built-in function compiles and runs whatever Python source code statements are in the string you pass to it. Along with its `eval` expression cousin, this supports many dynamic roles you'll meet in upcoming chapters.

By passing a code file's loaded contents to `exec`, though, this yields another way to launch code files from a REPL or other file—without having to import and later reload. Each such `exec` runs the current version of the code read from a file, without requiring imports or reloads. For instance, using Example 3-1's `script1.py` again:

```
$ python3 >>> exec(open('script1.py').read()) darwin
1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!
```

To understand this code, you must first know that the `open` and `read` nested inside it run first, and left to

right, and load the file's entire contents as a string. We'll study files in the next part of this book, so take this as a preview for now. (This was also a forward knowledge dependency, but that's what this book presents.)

Once the file is loaded, though, the `exec` call has an effect similar to an `import`, but it doesn't actually import the module. Instead, each time you run an `exec/open` combo this way it runs the file's code anew and unconditionally—as though you had pasted the file's code at the place where `exec` is called. As one consequence, this `exec/open` scheme does not require module reloads after file changes.

On the downside, because it works as if you've pasted code into the place where it is called, `exec` has the potential to silently overwrite variables you may currently be using. For example, our `script1.py` assigns to a variable named `x`. If that name is also being used in the place where `exec` is called, the name's value is replaced—sans warning:

```
>>> x = 999 >>> exec(open('script1.py').read()) # Code run in this namespace by default...same output...
>>> x # Its assignments can overwrite names here'Hack!'
```

This potential for name collisions is a downside shared by the last section's `from` statement. By contrast, the basic `import` statement makes the file a separate

module namespace, so that its assignments will not change variables in the importer's scope. The price you pay for its namespace partitioning of modules is the need to reload and qualify.

中文翻译

Python 还提供了一种不依赖上一节模块概念、用代码启动文件的方式。exec 内建函数会编译并运行你传给它的字符串里的任何 Python 源代码语句。连同它的表兄弟——表达式求值的 eval——一起，支撑着你在后续章节会遇到的许多动态 (dynamic) 角色。

不过，把代码文件加载后的内容传给 exec，就得到了从 REPL 或其它文件启动代码文件的另一种方式——无需 import、之后也无需 reload。每一次这样的 exec 都会运行从文件读到的当前版本的代码，不需要导入或重载。例如，再用示例 3-1 的 script1.py：

```
$ python3 >>> exec(open('script1.py').read()) darwin
1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!
```

要理解这段代码，你得先知道：嵌套在里面的 open 和 read 先执行，从左到右，把文件的全部内容加载成一个字符串。我们会在本书下一部分学习文件，这里就先当作一次预览吧。

一旦文件被加载，exec 调用的效果就与 import 类似，但它并不真正导入模块。相反，你每运行一次这样的 exec/open 组合，它都会重新、无条件地运行文件的代码——就好像你把文件的代码粘贴在调用 exec 的位置一样。一个直

接后果是：这种 `exec/open` 方案在文件修改后不需要重载模块。

缺点在于：因为它的行为如同把代码粘贴在调用处，`exec` 有可能静默覆盖你当前正在使用的变量。例如，我们的 `script1.py` 给名为 `x` 的变量赋值。如果这个名字在调用 `exec` 的地方也正被使用，这个名字的值就会被替换——没有任何警告：

```
>>> x = 999 >>> exec(open('script1.py').read()) # 代码默认在这个命名空间里运行...同样的输出...>>> x # 它的赋值可以覆盖这里的名字'Hack!'
```

这种名字冲突的可能性，是上一节的 `from` 语句同样存在的缺点。与之相反，基础的 `import` 语句会把文件变成独立的模块命名空间，使它的赋值不会改变导入者作用域里的变量。模块命名空间隔离的代价，就是你必须 `reload` 和点号限定。

深度理解

·核心概念：`exec` = 在调用处直接执行一段代码字符串。`exec(open('file.py').read())` = "把文件内容当成粘贴过来的代码运行"，是 `import` 之外的另一种"运行文件"手段。

·底层机制：`open('script1.py')` 返回文件对象 → `.read()` 读出整个文件内容字符串 → `exec` 接收字符串，现场编译成字节码并在当前命名空间执行。与 `import` 的关键区别：`exec` 不查 `sys.modules`、不做缓存、不产生模块对象——每次都是"全新执行"。

·为什么能免 `reload`：因为 `exec` 每次都是"读最新文件内

容再执行", 天然拿到的就是当前版本。import 的缓存机制反而成了需要 reload 的根源。

·为什么危险: exec 的代码在调用者的命名空间里运行——文件里赋值 `x = 'Hack!'` 会把调用处已有的 `x = 999` 静默覆盖。这正是 Python 哲学里的"显式优于隐式"的反面教材: 隐式注入名字 = 隐患。

·对比三兄弟: import (隔离命名空间, 安全但需 reload + 限定)、from (复制名字, 省事但不同步)、exec (粘贴代码, 最灵活但最危险)。三者是"隔离性 → 便利性"的光谱。

·现实用途: exec 常被用于配置文件加载 (读一段 Python 语法配置)、插件系统、DSL 执行。生产代码里要谨慎——exec 任意字符串等于运行任意代码 (安全风险)。

·初学者误区: ①以为 exec 之后还能 reload——exec 与模块系统无关; ②以为 exec 不会改当前变量——恰恰相反, 它默认就在当前命名空间执行; ③把 exec 与 eval 混用——exec 运行语句、eval 求值表达式。

代码分析

```
exec(open('script1.py').read()) # →
x = 999                          #
exec(open('script1.py').read()) # script1.py  x='Hack!' → ...
x                               # 'Hack!'
```

解释器如何处理:

1. `open('script1.py')`: 打开文件, 返回文件对象 (Python 的文件操作, Part II 讲)。

2. `.read()`: 一次性读出全部文本, 返回 `str` 对象。
 3. `exec(字符串)`: CPython 对这个字符串调用编译管线 (`tokenize` → `parse` → 字节码), 然后在当前帧的命名空间逐条执行——相当于“文本粘贴执行”。
 4. 副作用: 文件里的 `x = 'Hack!'` 直接改写调用处命名空间的 `x`, 覆盖掉原来的 999。
 5. 设计思想: `exec` 是 Python “一切皆可动态”哲学的极致体现——连“程序文本”本身都可以是运行时输入。能力越大责任越大: 生产环境慎用。
-

3.11.5 命令行启动器 (Command-Line Launchers)

英文原文

Finally, Python code can also be run by Python code that uses standard-library tools to spawn command lines in parallel processes. The `os` module's `system` call, for instance, runs a command as if you typed it at a system console (its 0 return value echoed at the end here means all worked well):

```
$ python3 >>> import os >>> os.system('python3 script1.py')
darwin 1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack! 0
```

This `os` module's `popen` call does similar but returns a file object from which you can read the spawned command's printed output as a string for use in the spa

wning code; and Python's subprocess.run call can be used to launch programs by command lines with much more control over the fine points. We'll deploy popen in Chapter 21.

Though related to running code, all these tools are well beyond the scope of this getting-started chapter, so consult Python's library manuals for the full story if and when such utilities become useful in your future work. And a caution: these tools, along with exec, will happily run any command line you throw at them—including one to erase all your files!—so they're best limited to running Python commands unless and until you're sure that other commands are safe.

中文翻译

最后，Python 代码还可以由使用标准库工具、在并行进程中生成命令行的 Python 代码来运行。例如，os 模块的 system 调用会像你在系统控制台输入一样运行一条命令（这里结尾回显的 0 是返回值，表示一切正常）：

```
$ python3 >>> import os >>> os.system('python3 script1.py') darwin 1267650600228229401496703205376 Hack!Hack!Hack!Hack!Hack!Hack!Hack!Hack! 0
```

os 模块的 popen 调用与此类似，但会返回一个文件对象，你可以从中把所启动命令打印的输出读成字符串，供启动它的代码使用；而 Python 的 subprocess.run 调用则能用命令行启动程序，并对细节有更多控制。我们会在第 21 章用到 popen。

虽然都与运行代码相关，但这些工具都远远超出本章这个入门章的范畴；如果这类工具在你将来的工作中变得有用，请查阅 Python 的库手册了解全貌。还有一条告诫：这些工具连同 `exec`，会乐此不疲地运行你丢给它们的任何命令行——包括一条能删光你所有文件的命令！——所以在你确信其它命令是安全的之前，最好只让它们运行 Python 命令。

深度理解

·核心概念：Python 程序可以自己启动外部进程——`os.system`（扔给 shell 执行）、`os.popen`（可读输出）、`subprocess.run`（现代推荐，精细控制）。这是“程序内的命令行”。

·底层机制：`os.system('python3 script1.py')` 调用操作系统 API 启动一个新的子进程，由系统的 shell 解析并执行命令字符串；子进程的 `stdout` 直接连到当前进程的 `stdout`（所以输出照常显示）；返回值 0 表示退出码为 0（成功）。

·为什么现代推荐 `subprocess`：`os.system` 把命令交给 shell 解析——一旦命令字符串含空格/特殊字符/用户输入，就有注入与转义风险；`subprocess.run` 支持参数列表传递（不经 shell）、超时、捕获输出、错误检查，是 `os.system` 的现代替代。

·安全告诫的分量：`os.system('rm -rf /')`（Unix 删文件）这类命令一旦执行就不可挽回。永远不要用这些工具执行未经检查的字符串——尤其是来自用户输入或网络的内容。

·生活类比：os.system 像"按下遥控器让机器人替你去敲键盘"——方便，但机器人不挑命令，你说删库它真删。

·初学者误区：①把 os.system 当"运行 Python 代码"的常规手段——它运行的是系统命令，不是 Python 语句；②以为返回值是输出内容——它是退出码；③在 Windows 上把 Unix 命令（如 ls）传给它。

代码分析

```
import os                                # os
os.system('python3 script1.py') # shell 0
os.popen('python3 script1.py') # system
import subprocess
subprocess.run(['python3', 'script1.py'], capture_output=True, tex...
```

解释器如何处理：

1. os.system(...): CPython 调用 C 库的 system() → 操作系统 fork 一个子进程，用 sh -c "python3 script1.py" 执行 → 子进程运行完，父进程拿到退出状态码（0 = 成功）。

2. os.popen(...): 也是启动子进程，但把子进程的 stdout 接回一个管道，包装成 Python 文件对象——你可以 .read() 读出全部输出。

3. subprocess.run([...]): 直接 exec 程序（不经过 shell 解析，所以参数用列表，避免转义问题）；capture_output=True 捕获输出；返回 CompletedProcess 对象。

4. 安全设计思想：任何"运行外部命令"的 API 都是潜在后门——Python 文档反复强调这一点，开发者必须自己守好"什么命令可以跑"的边界。

3.12 其它启动选项 (Other Launch Options)

英文原文

You've already seen a blizzard of code-launching options for Python, but this accounting is still incomplete. For example, Python code can also be run today by:

- Programs written in C, C++, Java, and more, which embed Python and Python code
- Text editors that aren't full IDEs, but know how to run Python code you're editing
- Excel spreadsheets, when calculating a sheet's cells coded in Python
- Web servers that spawn scripts automatically in response to browser UI actions
- Users launching standalone executables, which we explored in Chapter 2
- And (as usual) more

Moreover, launching techniques tend to evolve as rapidly as everything else in computing, and future options are impossible to foresee. In general, because Python keeps pace with such changes, you should be able to launch Python programs in whatever way makes sense for the machines you use, both now and in the future—be that by swiping on your smartphone, grabbing icons in a virtual reality, or shouting a script's name over your coworkers' conversations.

中文翻译

你已经看到了铺天盖地的 Python 代码启动选项，但这个盘点仍然不完整。例如，今天 Python 代码还可以由下面这些方式运行：

- 用 C、C++、Java 等语言编写的程序，它们嵌入 (embed) 了 Python 和 Python 代码- 不是完整 IDE、但懂得如何运行你正在编辑的 Python 代码的文本编辑器 (editors)
- Excel 电子表格——当用 Python 编写、计算工作表中单元格的时候- Web 服务器 (servers) ——响应浏览器 UI 操作自动生成脚本- 用户启动独立可执行文件 (standalone executables) ——这在第 2 章探讨过- 还有 (一如既往地) 更多

而且，启动技术往往和计算世界的其它一切一样快速演变，未来的选项无法预见。总体而言，因为 Python 跟得上这些变化，你现在和未来都应该能按你所用的机器上最合理的方式启动 Python 程序——无论是在智能手机上滑动、在虚拟现实里抓图标、还是在同事们的聊天声里喊出脚本的名字。

深度理解

- 核心概念：Python 的启动方式远不止本章讲的这些——嵌入 (embedding)、编辑器、Excel、Web 服务器、独立可执行文件.....这个清单本身"不可穷尽"。

- "嵌入"意味着什么：C/C++/Java 程序可以把 Python 解释器当库链接进来，在自己的进程里运行 Python 代码 (CPython 提供 Python.h C API, Jython 则相反，是 J

ava 调用 Python)。这是"Python 作为脚本语言被宿主程序驱动"的经典模式。

- Web 服务器场景：像 Django/Flask 等框架的部署，就是 Web 服务器在收到浏览器请求时自动运行 Python 代码——你"点按钮"，服务器"跑脚本"。

- 为什么单子总写"以及更多"：启动技术随时代演化（智能手机滑动、VR 抓图标……）——作者借这段话传递"拥抱变化"的态度：先掌握通用的核心机制（命令行/REPL），任何新 UI 都只是同一解释器的不同外壳。

- 生活类比：Python 解释器像"通用发动机"——可以装在汽车（命令行）、摩托（App）、甚至航天飞机（嵌入系统）上，关键是发动机本身没变。

- 初学者误区：①以为必须记住所有启动方式——不需要，知道"解释器 + 代码"这个本质即可，新花样来了自然懂；②看到"嵌入"就以为 Python 不能独立运行——恰恰相反，独立运行才是主流。

3.13 该用哪个选项？ (Which Option Should I Use?)

英文原文

With all these options, beginners might naturally ask, Which one is best for me? In general, you should try both basic command lines and the IDLE GUI if you are just getting started with Python, unless you'll be w

orking in smartphone apps or web-based notebooks with interfaces that are more unique. Command lines are simple and powerful, but IDLE provides a user-friendly GUI environment that hides some underlying details and works the same on all PCs.

If, on the other hand, you are an experienced programmer, you might be more comfortable with simply the text editor of your choice in one window, and a system console interface in another, for launching edited programs via Python command lines. Because the choice of development environments is very subjective, this book can't offer much more in the way of universal guidelines. In general, whatever environment you like to use will be the best for you to use.

中文翻译

面对这么多选项，初学者自然会问：哪个最适合我？总的来说，如果你是刚开始学 Python，应该把基础命令行和 IDLE GUI 都试试，除非你将在界面更独特的手机应用或网页笔记本里工作。命令行简单而强大，而 IDLE 提供了一个用户友好的 GUI 环境，隐藏了一些底层细节，并且在所有 PC 上表现一致。

反之，如果你是有经验的程序员，你可能更习惯这样的配置：一个窗口放着顺手的文本编辑器，另一个窗口是系统控制台界面，用 Python 命令行启动刚编辑好的程序。因为开发环境的选择非常主观，本书无法给出更多放之四海的准则。总的来说，你喜欢用的环境，就是最适合你的环境。

深度理解

·核心概念：这是全章的选择题答案——"没有唯一正确答案，环境选择极其主观"。作者给出两条建议：新手试"命令行 + IDLE"；老手用"编辑器 + 控制台"。

·为什么推荐新手双轨制：命令行暴露"命令是什么、参数是什么、目录在哪"的底层事实；IDLE 提供"点按钮就跑"的友好体验。两者互为补充：一个给你原理，一个给你舒适区。

·"Whatever environment you like"的深意：开发工具不决定你的编程能力，你能理解代码如何被执行才决定。工具只是外壳，解释器才是核心——所以作者敢说"你喜欢的就是最好的"。

·实际影响：选择标准 = 稳定性（不折腾环境）+ 可见性（能看到错误信息）+ 可重复性（配置可迁移）。别频繁换 IDE，工具换来换去才是最耗时的。

·生活类比：写文章用 Word、记事本还是 Markdown？——能写出好文章的人用什么都行，工具之争远小于内容之争。

·初学者误区：①花大量时间"选 IDE"而不写代码——工具选择适可而止；②以为"别人推荐的一定适合我"——自己动手试过才知道。

3.14 本章小结 (Chapter Summary)

英文原文

In this chapter, we've explored common ways to launch Python programs: by running code typed interactively and by running code stored in files with system command lines, file icon clicks, IDE GUIs such as IDLE, and more. We've covered a lot of pragmatic startup territory here. This chapter's goal was to equip you with enough information to enable you to start writing some code, which you'll do in the next part of this book. There, we will start exploring the Python language itself, beginning with its core data types—the objects that are the subjects of your programs.

First, though, take the usual chapter quiz to exercise and review what you've learned here. Because this is the last chapter in this part of the book, it's followed with a set of more complete exercises that test your mastery of this entire part's topics. For help and answers for the latter set of problems, or just for a refresher, be sure to turn to Appendix B after you've given the exercises a try.

中文翻译

本章我们探索了启动 Python 程序的常见方式：交互式地运行输入的代码、用系统命令行运行存储在文件里的代码、点击文件图标、使用 IDLE 等 IDE GUI，以及更多方式。我们覆盖了大量务实的启动知识。本章的目标是让你掌握足够的信息，能够开始编写一些代码——这正是你将在本书下一

部分要做的事。在那里，我们将开始探索 Python 语言本身，从它的核心数据类型（data types）开始——那些将成为你程序主体的对象。

不过，先照例做一下本章测验（quiz），练习并复习你在这里学到的内容吧。因为本章是本书这一部分的最后一章，它后面还跟着一套更完整的练习，用于测试你对整个部分主题的掌握程度。对于后一组练习，需要帮助或答案，或者只是想复习一下，请在尝试过练习之后务必翻阅附录 B。

深度理解

·核心概念：第一部分收官——"启动"专题全部讲完：交互式 REPL、脚本文件、命令行变体、图标点击、IDLE、其它 IDE、App、WebAssembly、Jupyter、AOT 编译器、import/reload/exec/命令行启动器。下一部分（第 4 章起）正式进入语言本身。

·为什么把"运行"放第一部分：因为 Python 是"实践驱动"的语言——你必须在第 4 章之前就能独立运行任何代码片段，否则后面所有示例都是纸上谈兵。作者确保你在"厨房"阶段就能开火做饭。

·本章的隐藏主线：一切启动方式最终都归结为"让解释器执行代码"——入口千万个，本质一个。带着这个视角看后面的学习，你会对"为什么 IDE 能运行我的代码"保持清醒。

·测验+练习的教学法：每章末尾的 Quiz 是"即时回忆测试"；第一部分末尾的 Exercises 是"综合运用测试"，答案在附录 B（部分答案还含正文没有的补充信息，作者鼓励你

即使做对了也翻一翻)。

·初学者误区：①跳过测验直接下一章——回忆本身就是最重要的学习环节；②只看答案不做题——练习的"试错过程"才是收获所在。

3.15 测验：测测你的知识 (Test Your Knowledge: Quiz)

英文原文

1. How can you start a Python interactive interpreter session (REPL)? 2. Where do you type a system command and line to launch a script file? 3. Name four or more ways to run the code saved in a script file. 4. What pitfall is related to clicking file icons on Windows and Linux? 5. Why might you need to reload a module? 6. How do you run a script from within the IDLE GUI? 7. How are modules, attributes, and namespaces related?

中文翻译

1. 你怎么启动一个 Python 交互式解释器会话 (REPL) ? 2. 你在哪里输入系统命令来启动一个脚本文件? 3. 说出至少四种运行脚本文件里保存的代码的方式。4. 在 Windows 和 Linux 上点击文件图标有什么陷阱? 5. 你为什么会需要重新加载 (reload) 一个模块? 6. 你怎么在 IDLE GUI 里运行一个脚本? 7. 模块、属性和命名空间之间是什么关系?

深度理解

·核心概念：7 道"本章体检题"，覆盖全章最值得记住的操作点：REPL 启动、命令行位置、启动方式枚举、图标点击陷阱、reload 动机、IDLE 操作、模块三概念。

·教学用意：这些问题全部是"能答上 = 能独立上手"的操作性知识，不涉及语法。作者故意放弃"概念题"而全用"操作题"，因为第一部分的目标就是让你能把代码跑起来。

·自测方法：试着不看书写出完整答案；卡住的地方回头找对应小节——这是主动回忆（active recall）最好的用法。

·初学者误区：①觉得题目"太简单"就跳过——正因为它简单，才说明你还没在真实环境里踩过这些坑；②只记概念不亲手操作——本测验的每个答案都需要你在电脑上试过才记得牢。

3.15.1 答案 (Answers)

英文原文

1. A Python interactive session can be started by typing a Python command line in your system's console window. Type `py` on Windows and `python3` everywhere else, and type it into a Command Prompt or Power Shell window on Windows, Terminal on macOS and Linux, and the Termux app on Android. Another alternative is to launch IDLE, as its main Shell window is an interactive session. Other IDEs, smartphone apps, an

d browser-based systems may offer REPLs in more unique ways.

2. You type system command lines in the same interface used to launch an interactive session by command line. This is whatever your platform provides as a system console: again, Command Prompt or PowerShell on Windows, Terminal on macOS and Linux, Termux on Android, or other. You type this at the system's prompt shown as `$` in this book, not at the Python interactive interpreter's `>>>` prompt used to enter Python code—be careful not to confuse these prompts, because Python and the system shell are different systems.

3. Code in a script (really, module) file can be run with system command lines, file icon clicks, imports and reloads, the `exec` built-in function, `os` module tools, and IDE GUI devices such as IDLE's Run → Run Module menu option. Some platforms support more specialized launching techniques, like drag and drop on macOS, app UIs on smartphones, and web notebooks. In addition, some text editors have unique ways to run Python code, some Python programs are provided and run as standalone executables, and some systems use Python code in embedded mode. Though in its early days, code may also be run in web browsers with a Python port to WebAssembly like Pyodide.

4. Scripts that print and then exit cause the output wi

ndow to disappear almost immediately, before you can view the output. `input()` can pause before `exit` so the output window stays open, but error messages generated by your script's code also appear in that window and can blank it before you can examine it. Hence, system command lines and IDEs are your best bet for most development.

5. Python imports (i.e., loads) a module only once per process, by default. If you've changed your module's source code and want to run the new version in the same process, you'll need to reload it. You must import a module at least once before you can reload it. Reloading is far less common than importing in practice—a more typical solution to "new code" needs is to restart the process or use an IDE/command line.

6. Within the text edit window that holds the script you wish to run, select the window's `Run` → `Run Module` menu option. This runs the window's source code as a top-level script file in IDLE, and displays its output back in the interactive Python "Shell" window.

7. Each module file is automatically a namespace—that is, a package of variables reflecting the assignments made at the top level of the file. Each of the module's variables is an attribute of the module, accessed by `"."` qualification or from name copies. The namespace concept is what makes module code reusable and keeps program code organized.

中文翻译

1. 可以通过在系统控制台窗口里输入一条 Python 命令行来启动 Python 交互式会话：Windows 上输 `py`，其它地方输 `python3`，并把命令输入到 Windows 的命令提示符或 PowerShell 窗口、macOS 和 Linux 的终端（Terminal）、Android 的 Termux 应用里。另一个替代方案是启动 IDLE，因为它的主 Shell 窗口本身就是一个交互式会话。其它 IDE、智能手机应用和基于浏览器的系统可能以更独特的方式提供 REPL。

2. 系统命令行输入的地方与“用命令行启动交互式会话”的界面相同——即你的平台提供的系统控制台：Windows 的命令提示符或 PowerShell、macOS 和 Linux 的终端、Android 的 Termux，等等。你要在本书中以 `$` 表示的系统提示符处输入，而不是在 Python 交互式解释器输入 Python 代码所用的 `>>>` 提示符处——务必小心不要把这两种提示符搞混，因为 Python 和系统 shell 是两个不同的系统。

3. 脚本（准确说是模块）文件里的代码可以通过系统命令行、点击文件图标、`import` 与 `reload`、`exec` 内建函数、`os` 模块工具、以及 IDLE 的运行 → 运行模块（Run → Run Module）等 IDE GUI 选项来运行。更多例子还包括 macOS 的拖放、智能手机的 App UI、网页笔记本等。此外，一些编辑器能一键运行当前文件，有些 Python 程序以独立的可执行文件形式分发运行，还有些系统以嵌入（embedded）方式运行 Python。另外，用 Pyodide 这类把 Python 移植到 WebAssembly 的方案，代码目前也能在浏览器里运行。

4. 只打印然后退出的脚本，会让控制台窗口在你能查看输出之前就几乎立刻消失。`input()` 可以让窗口在退出前暂停、保持可见，但脚本生成的错误消息也会出现在那个窗口里，而且这些错误信息可能还没等你看到就消失了。因此对大多数开发来说，系统命令行、IDE 才是更稳妥的选择。

5. 默认情况下，Python 在每个进程里只导入（即加载）一个模块一次。如果你改了模块源码，又想在同一个进程里运行新版本，就必须 reload 它。至少成功 import（或 from）一次后你才能 reload。实践中 reload 远不如 import 常用——更常见的“换新代码”做法是重启进程，或直接使用 IDLE/命令行重新运行。

6. 在保存了你想要运行的文件的文本编辑窗口里，选择窗口菜单的运行 → 运行模块（Run → Run Module）。这个命令会把窗口里的源代码作为顶层脚本文件在 IDLE 中运行，并把输出显示在同一个交互式 Python "Shell" 窗口里。

7. 每个模块文件自动就是一个命名空间（namespace）——即反映文件顶层赋值的一个变量包。模块的每个变量都是模块的一个属性（attribute），可通过"." 点号限定或 from 名字复制来访问。命名空间这个概念保证模块组织有序且程序各部分能协调一致。

深度理解

·答案 1：核心信息是“命令因平台而异”——`py`（Windows launcher）/ `python3`（其它平台）；同样重要的还有“控制台也要对应平台”（Command Prompt/Terminal/Termux）。

- 答案 2：重新强调两套提示符不能混：\$ 是系统 shell，>>> 是 Python——这是新手事故第一高发地。
 - 答案 3：这道题的考点是"启动方式多态"——命令行、图标、导入、exec、os 工具、IDE、浏览器全能跑，答案只要求 4 种以上，背不全也不要紧，掌握"导入=执行"这一主线即可。
 - 答案 4：呼应 3.10.1——闪退的本质是"输出窗口生命周期"问题，input() 只能救输出、救不了闪退的错误消息。
 - 答案 5：点名 reload 的两个前提：先 import 才能 reload；reload 是低频操作——工程上"重启进程"更普遍。
 - 答案 6：IDLE 的标准流程 = 编辑窗口里 Run → Run Module，对应 3.10.2 的讲解。
 - 答案 7：三词缝合——模块=命名空间，命名空间里的名字=属性；访问方式两种（.点号或 from 复制）。这是 Part V 的地基。
-

3.16 练习：测验你的知识 (Test Your Knowledge: Part I Exercises)

英文原文

It's time to start doing a little coding on your own. This first exercise session is fairly simple, but it is designed to make sure you're ready to work along with the rest of the book, and a few of its questions hint at topics to come in later chapters. Be sure to check the s

ection "Part I, Getting Started" in Appendix B for the answers; the exercises and their solutions sometimes include supplemental information not discussed in the main text, so you're encouraged to take a peek at the solutions even if you manage to answer all the questions on your own.

1. Interaction: Using a system command line, IDLE, or any other method that works on your device, start the Python interactive command line (the `>>>` prompt, a.k.a. the REPL), and type the expression `'Hello World!'` (including the quotes). The string should be echoed back to you. The purpose of this exercise is to get your environment configured to run Python.

2. Programs: With the text editor of your choice, write a simple module file containing the single statement `print('Hello module world!')` and store it as a file named `module1.py`. Now, run this file by using any launch option you'd like: running it in IDLE, clicking or tapping on its file icon, starting it by command line in a console (e.g., `python3 module1.py`), executing it in code with tools like `exec` and `imports/reloads`, or by using UI options in apps, other IDEs, and web notebooks. In fact, experiment by running your file with as many of the launch techniques discussed in this chapter as you can. Which technique seems easiest to you?

3. Modules: Start the Python interactive command line (the `>>>` prompt) and import the module you wrote

e in exercise 2. Try moving the file to a different directory and importing it again from its original directory. What happens? Hint: there's still a .pyc bytecode file for module1 in a pycache subdirectory there, but it's named oddly. In general, imports import from the current directory, plus every directory listed on the PYTHONPATH environment variable.

4. Scripts: If your platform supports it, copy your module1.py module file to another named script1.py; then add the #! line to the top of script1.py, give this file executable privileges, and run it directly as an executable (e.g., sans the "python3"). What does the first line need to contain? #! lines traditionally have meaning only on Unix-like platforms, but also apply to Windows today, thanks to the Windows launcher.

5. Errors and debugging: Experiment with typing mathematical expressions and assignments at the Python interactive command line (REPL). Along the way, type the expressions 2 500 and 1 / 0, and reference an undefined variable name as we did early in this chapter. What happens? You may not know it yet, but when you make a mistake, you're doing exception processing—a topic we'll explore in depth in Part VII. A Python error is actually a caught exception.

6. Breaks and cycles: At the Python interactive command line (REPL), type: >>> L = [1, 2] # Make a 2-item list >>> L.append(L) # Append L as a single item to it

self >>> L # Print L: a cyclic/circular object What happens? In all but the most ancient of Python versions, you'll see a strange output that is described in the Appendix B solution, and which will make more sense when you study object references in the next part of the book.

7. Documentation: Spend at least five minutes browsing the Python library and language manuals before moving on to get a feel for the available tools in the standard library and the structure of the documentation. You can find these manuals via the Start-menu entries on Windows, the FAQ and other resources on the web, and in Python's official docs. You'll also learn more about the manuals and other documentation sources, including PyDoc, in Chapter 15.

中文翻译

是时候开始自己动手写一点代码了。这第一次的练习很简单，但设计目的是确保你准备好跟着本书的其余部分一起动手；其中几个问题还暗示了后面章节会出现的主题。务必查看附录 B 中的 "第一部分，入门" 一节核对答案；练习及其解法有时会包含正文没有的补充信息，所以即使你能独自答对所有问题，也鼓励你去瞄一眼答案。

1. 交互 (Interaction)：用系统命令行、IDLE 或任何在设备上凑效的方法，启动 Python 交互式命令行 (>>> 提示符，也叫 REPL)，输入表达式 'Hello World!' (包括引号)。字符串应该被回显给你。这个练习的目的是把你的环境配置好，能跑 Python。

2. 程序 (Programs) : 用你顺手的文本编辑器, 写一个只包含单条语句 `print('Hello module world!')` 的简单模块文件, 存成名为 `module1.py` 的文件。现在用任意你喜欢的启动方式运行它: 在 IDLE 里运行、点击/轻点文件图标、在控制台命令行启动 (例如 `python3 module1.py`)、用 `exec`、`import/reload` 等工具在代码中运行它, 或在 App、其它 IDE、网页笔记本里用 UI 选项运行。事实上, 尽量把本章讨论过的多种启动技术反复测试, 看看哪种对你来说最容易。

3. 模块 (Modules) : 启动 Python 交互式命令行 (`>>>` 提示符), 导入你在练习 2 里写的模块。试着把文件移到另一个目录, 再从原目录导入它, 会怎样? 提示: 那里有一个 `module1` 的 `.pyc` 字节码文件留在 `pycache` 子目录里, 但它的名字看起来怪怪的。总的来说, 导入会扫描当前目录, 再加上环境变量 `PYTHONPATH` 列出的每个目录——正如你将在第五部分学到的。

4. 脚本 (Scripts) : 如果你的平台支持, 把 `module1.py` 复制一份命名为 `script1.py`, 然后在 `script1.py` 顶部加一行 `#!` 开头的行, 给这个文件可执行权限, 并把它当可执行程序直接运行 (即不带 `"python3"`)。第一行需要包含什么? `#!` 行传统上只在类 Unix 平台上有意义, 但如今在 Windows 上也同样适用——这要感谢 Windows 的 `py launcher`。

5. 错误与调试 (Errors and debugging) : 在 Python 交互命令行 (REPL) 里输入数学表达式和赋值语句做实验。途中输入 `2 500` 和 `1 / 0` 两个表达式, 并像本章前面那样引用一个未定义的变量名。会发生什么? 你可能还不知道,

但当你犯错时，你其实已经无意中接触到了异常处理（exception handling）——一个我们会在第七部分深入探讨的主题。从技术上说你触发的是所谓的默认异常处理器（default exception handler）——打印标准错误消息的逻辑。如果不捕获错误，默认处理器就会接住它并打印标准错误消息来回应。异常和 Python 里的调试（debugging）概念也密切相关：刚开始时，Python 默认的错误消息大概就能提供你需要的全部错误处理支持——它们既指出错误原因，也显示出错时活跃的代码行。更多调试内容请看本章前面的侧边栏“调试 Python 代码”（Debugging Python Code）。

6. 环与循环（Breaks and cycles）：在 Python 交互环境（REPL）里输入：

```
>>> L = [1, 2] # 创建一个两个元素的列表
>>> L.append(L) # 把 L 作为单个元素追加进它自己
>>> L # 打印 L：一个循环环（circular）对象会发生什么？
```

除了最古老的 Python 版本外，你会看到一个奇怪的输出——这个输出在附录 B 的答案里有解释，等你们学到下一部分的对象引用（object references）时会明白。想想看，为什么你的 Python 会这样回应？

7. 文档（Documentation）：继续前进之前，至少花五分钟浏览 Python 的库手册和语言手册，感受一下标准库有哪些可用工具、以及整个文档集的结构。要熟悉手册集中主要主题的位置，这个时间至少是必要的；一旦做过一次，之后找自己需要的东西就很容易了。这类手册可以通过 Windows 开始菜单的 Python 条目、IDLE 的“帮助”下拉菜单里的“Python Docs”选项、或者 python.org 在线的 <http://www.python.org/doc> 找到。第 15 章你还会学到更多关于手册集和其它文档来源的内容——包括 PyDoc 和 help 函数

。如果还有时间，去逛一逛 Python 网站，以及它的 PyPI 第三方扩展库（python.org 的 "About" 和搜索功能在你起步时可能都挺有用）。

深度理解

·定位：这 7 道练习是"第一部分结业考"，覆盖本章全部技能，但题目里埋了三个"未来预告"：①模块与 .pyc（Part V）；②#! 与可执行脚本（Unix/Windows 差异）；③异常与调试（Part VII）。做练习时遇到不懂的概念不用慌——答案在附录 B，概念在后面的部分。

·练习 1（交互）：验证环境能跑通 REPL——这是后续所有章节的"开机自检"。

·练习 2（程序运行）：同一个文件用尽量多的启动方式跑——这是把本章知识"内化"的最佳方法，跑完后你会发现：启动方式虽多，解释器只有一个。

·练习 3（模块移动）：揭示 pycache 的存在与命名规则（.pyc 名称含解释器版本和源文件哈希）——把文件移走后，旧字节码还躺在 pycache 里，名字却对不上，这就是"放错目录"的活标本。

·练习 4（脚本）：#! shebang 行的本质是给操作系统看的"用哪个解释器"声明，加上可执行权限后可直接 ./script1.py 运行——只在 Unix 系平台有意义，Windows 由 py launcher 接管。

·练习 5（报错）：亲手"制造"异常并观察 traceback，理解"报错不崩溃+错误消息是朋友"。输入 2 500 是故意引导你体验 Python 大整数无溢出。

·练习 6 (列表环) : L.append(L) 造出一个指向自身的循环引用列表, 打印出的 [...] 省略号是 Python 防止无限递归显示的自我引用标记——这是学习"引用 (references)"的绝佳实验。

·练习 7 (文档) : 养成"查官方文档"的习惯, 为后续学习提供"救生圈"。

·练习方法: 先独立完成, 再对照附录 B; 即使全对也建议看答案——答案常含正文没有的补充信息 (作者原文如此) 。

3.17 侧边栏: 调试 Python 代码 (Debugging Python Code)

英文原文

Naturally, none of this book's readers ever have bugs in their code (insert smiley here), but for those less fortunate friends of yours who may, here's a quick review of the strategies commonly used by real-world Python programmers to debug errors in their code. The first two may be all you'll need early in the learning process, but others may grow important when you start writing larger scripts, and all are useful to review now before you start coding in earnest in the next chapter—and making the mistakes that are inevitable in programming:

Do nothing. This might read as odd, but when you make a mistake in Python, you get a useful and readable error message. The message pinpoints the location of the error in your code by file and line numbers and often offers suggestions about how to fix it. And if Python is new to you, that may be all you need—read the error and fix it.

Insert print statements. The most common debugging method in Python is to add temporary print statements at strategic places in your code and run it again, printing out the values of variables and other state at the points you're curious about. Because Python code runs immediately after changes, this is usually the fastest way to get a read on what's happening.

Use `pdb`. Python also ships with a debugger on a module named `pdb`, the Python debugger. Like the debuggers in IDEs, `pdb` lets you step through your code line by line, examine variable values, and so forth. You can launch it with `python3 -m pdb script.py`, or set `breakpoint()` in your code to open it right at that spot.

Insert calls to the logging module. If you also need permanent records of your program's progress, the logging module can route informational messages to files, syslog, and other places, with levels of severity including `DEBUG`, `INFO`, `WARNING`, and `ERROR`.

Use GUI debuggers. IDE GUIs such as IDLE include debuggers that let you step through code visually, set b

reakpoints with a click, and watch variable values, all without the command line. This can be easier for beginners than pdb.

Use the `-i` command-line flag. Short of adding prints or running a debugger, you can still get more info on errors: pass the `-i` flag to the interpreter when running a script, e.g., `python -i title.py`, to drop into the interactive prompt after the script completes, where you can inspect final variable values and continue exploring.

Record to the logging facility. This is not just print statements, but Python's library-managed logging with tools for message routing, level filtering, formatting, and more that can be very useful in larger programs.

Perhaps the most important thing to know about debugging in Python is that errors are detected and reported, not ignored. When you make a mistake, Python runs an error handler that prints a useful error message. This differs from some other languages, where you might only get a stack trace or nothing at all. Use this to your advantage—the error is your friend.

中文翻译

当然，本书的读者都从不在代码里出 Bug（此处插入笑脸表情）。但考虑你那些没那么走运的朋友，下面快速盘点一下真实世界 Python 程序员常用的调试策略。学习初期，前两个可能就够你用了；等你开始写较大脚本时其它策略会变

得重要。它们现在都值得过一遍——免得你下一章开始认真写代码时，做出编程中难免会犯的错误：

什么都不做。这听起来奇怪，但：当你在 Python 里犯错时，得到的是一个既有用又可读的错误消息，它通过文件名和行号精准定位代码里的错误位置，有时还会给出建议。如果错误足够明白，这可能就是你需要的一切——读错误，修代码。

插 print 语句。Python 程序员最常用的调试方法就是插入 print 语句：在关注的代码处纷纷打印出变量值，然后再跑一次。因为 Python 改了代码后立刻就能运行，这通常是最快获得“现场情报”的方法。

用 pdb。Python 自带一个命令行调试器，模块叫 pdb。它和 IDE 里的调试器原理类似：你可以单步执行代码、查看变量值等等。启动方式是 `python -m pdb 文件.py`，或在代码里写了 `breakpoint()` 的地方恰好在那一行进入调试器。

用 logging 模块。如果你还需要程序运行过程的持久记录，logging 模块能把信息类消息路由到文件、syslog 等处，并有轻重级别（DEBUG、INFO、WARNING、ERROR）。

用 GUI 调试器。IDLE 这样的 IDE GUI 带有调试器，可以可视化地单步通过代码、单击设置断点、观察变量值——对初学者比 pdb 容易上手。

用 -i 命令行开关。不加 print 也不跑调试器，你仍能窥探错误后的状态：运行脚本时加 -i 参数（如 `python -i title.py`），脚本运行结束或报错后，Python 会交互式地停在 >

>> 提示符处——在那里你可以查看最后的变量值、继续探索。

把一切都记录进日志（相关于 logging）。我们要鉴别一下：print 调试是即时的手工侦察，logging 是“制度化”的运行记录——logging 库提供的分级（级别）、路由、格式和数据完整性让它在生产级、大数据程序里更越好。

关于 Python 调试最重要的一句话：错误是被报告出来，而不是被悄悄吞掉。犯错时，Python 会运行一个异常处理器，打印有用的错误信息。这和某些会在错误时沉默的语言对比鲜明——善用它：错误消息是你的朋友。

深度理解

·核心概念：调试第一公理——Python 报错是“报告式”的，是对你友好。错误消息带文件名、行号、异常类型和描述，还能给出修正建议。把它当朋友而不是敌人。

·策略排序（学习曲线从低到高）：do nothing（读错）→ print（快而糙）→ logging（正式化）→ pdb（命令行单步）→ IDE GUI 调试器（可视化）→ -i（事后现场）→ try/except（主动捕获，Part VII）。初学用前两个就够了，生产环境用 logging + 调试器。

·pdb 与 breakpoint() 的底层：python -m pdb script.py 相当于“把调试器接在解释器上”；代码里的 breakpoint() 则在你指定的位置召唤调试器。Python 3.7+ 的 breakpoint() 是 pdb.settrace() 的现代语法。

·logging 为什么比 print 高级：print 只能输出到 console，项目发布后你没法“人肉盯屏”；logging 支持级（DE

BUG/INFO/WARNING/ERROR) 、路由、格式化、持久化——是"管理输出"的工程方案。

· -i 的现实意义：python -i script.py 跑完脚本不退出，直接进入 REPL——你可以打印最终变量值、甚至继续执行代码。这是"事后现场勘查"的利器，比一次次加 print 快。

·生活类比：报错如山上的"烟雾报警器"——它会告诉你位置（哪一行）、火的类型（异常类型），还可能提示你"是不是忘了关灶台"（did-you-mean 建议）。你要做的是读懂，然后灭火。

·初学者误区：①以为报错说明"我写的东西没救"——恰恰相反，Python 的错误消息是给你设计最贴心的地方；②只盯着最后一行看——要自下向上读 traceback，最顶层才是异常真正源头；③遇到错误就慌着删代码——先读错误、再打印、再断点，一步一步来。

本章总结

技术拓展 (Technical Expansion)

实际应用

·开发环境矩阵：本章的全部内容就是一份"环境排障手册"——现实中 90% 的 Python 新手卡在的不是语法，而是：python 不在 PATH、在 >>> 里输 shell 命令、脚本双击后窗口闪退、import 找不到同目录文件。把这些启动

技术练熟，等于修好了"环境-代码"之间的所有接口。

· Python 的"通用启动协议": 无论 IDE、Jupyter、Pyodi de 还是嵌入式 C 程序，底层都是同一件事——把源码交给解释器执行。因此你只需要掌握命令行（显式、可移植、可脚本化），任何新出现的 UI 都能秒懂。

·工程实践要点:

·用 `py -m script.py` 或 IDE 运行脚本，避免双击闪退。

·项目用"按章节/按模块"的目录组织，import 零配置可用

。

·程序内启动子进程优先用 `subprocess.run`，而不是 `os.system`。

·生产环境调试用 logging，临时排查用 `print + -i`，深入定位用 pdb/IDE 断点。

与 Java/C++ 的对比

维度	Python	Java	C/C++
启动方式	REPL、命令行、IDE、图标、import、exec、App、Web.....百花齐放	需先 javac 编译出 .class，再 java 启动 JVM	需先 gcc 编译链接出二进制，再执行
交互式运行	原生支持（REPL 即语言的一部分）	需 JShell (Java 9+) 等附属工具	无原生 REPL（可用 CINT/ROOT 等第三方）
编译步骤	通常无（字节码在内存/pycache 中）	必选（编译+打包+JVM）	必选（编译+链接）
模块/导入	.py 文件天然就是模块，import 即运行	package/import 需类路径（classpath）配置	头文件+链接器，模块化较繁琐

学习曲线	低（入门章就能跑起来）	中（环境+语法都要学）	高（内存、指针、编译链）
程序内运行代码	exec/eval/importlib 原生支持	反射（Reflection）较繁琐	几乎没有动态执行能力

Python 发展历史背景

- py 启动器：Windows 上的 py.exe 是 2011 年（PEP 397）随 Python 3.3 引入的，目的是解决多版本共存问题——同一台机器可以装 2.x 和 3.x，py -3 选择版本。
- reload 的三次变身：内置函数（早期）→ imp 模块（Python 2.x/3.0）→ importlib.reload（3.4，PEP 451 后）——imp 在 3.12 中被正式移除。
- IDLE 的历史：由 Guido van Rossum 本人早期参与设计，随 CPython 分发的"官方入门 IDE"，用 tkinter 写成——它证明了"Python 写 GUI"这件事本身可行。
- WebAssembly 时代：2022 年 CPython 官方仓库开始支持 Wasm 构建；Pyodide 从 2019 年起持续演进——"浏览器里跑 Python"从实验变成了可用的现实。
- AOT 编译的尝试：Nuitka（2015 年起）和 Shed Skin 走的是"Python 到 C 再到机器码"路线，说明 Python 社区从未停止对性能的追求。

高级开发者知识

- sys.path 与模块搜索机制：import 的搜索顺序 = 当前目录（脚本目录）→ PYTHONPATH → 标准库 → site-packages。理解它，才能诊断"import 不到"的所有问题。
- pycache 与字节码缓存：.pyc 文件的命名含 Python 版

本和哈希（如 `script1.cpython-312.pyc`），解释器靠它判断缓存是否失效。

·命名空间与作用域：模块 = 命名空间 = dict；`from ... import` 是复制、`import ...` 是绑定——这个区别在大型项目里决定“改一处是否生效”。

·子进程与 shell 注入：`os.system` 把字符串交给 shell 解析，存在注入风险；`subprocess.run([...])` 传参列表则无此问题——安全编程的分水岭。

·`-i`、`breakpoint()`、`PYTHONBREAKPOINT`：现代 Python 调试三件套，配合 `PYTHONBREAKPOINT` 环境变量可以把 `breakpoint()` 重定向到任何调试器（如 `pdb`、`debugpy`）。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) ——本部分是“实操地基”，运行方式不熟，后面所有章节的示例都无法亲手验证。

·应该掌握到什么程度：

1. 能手写命令行启动 REPL 和脚本（含 Windows `py` 与 Unix `python3` 的区别）；
2. 知道 `>>>` 和 `$` 是两种不同提示符，`#` 是注释；
3. 能说出至少 4 种运行脚本的方式（命令行/图标/IDLE/`import/exec`）；
4. 理解 `import` 的“执行文件 + 绑定模块对象”本质，以及为什么需要 `reload`；

- 5. 遇到报错能读 traceback、会用 print/-i/pdb 排查；
 - 6. 不必精通：Pyodide、AOT 编译器、Jupyter 的详细配置——知道"有这些选项"即可。
- 动手建议：完成本章末尾的 7 个练习（尤其第 6 题"列表循环引用"——它会在下一部分学习对象引用时成为最重要的直觉训练）。
- 后续应该学什么：第 4 章起进入第二部分——从核心数据类型（数字、字符串、列表、字典）开始正式学习语言；同时带着"模块/命名空间"的视角，在学到 Part V 时回望本章的 import 内容会豁然开朗。